

# Design & Development of Applications based on Micro Service Architecture

MSA

Vraj Suratwala

Department of ICT, VNSGU

# An Overview of Current Design Architectural Patterns

Introduction to Various Architectures

MSA

## 1.1.1 Monolithic Architecture

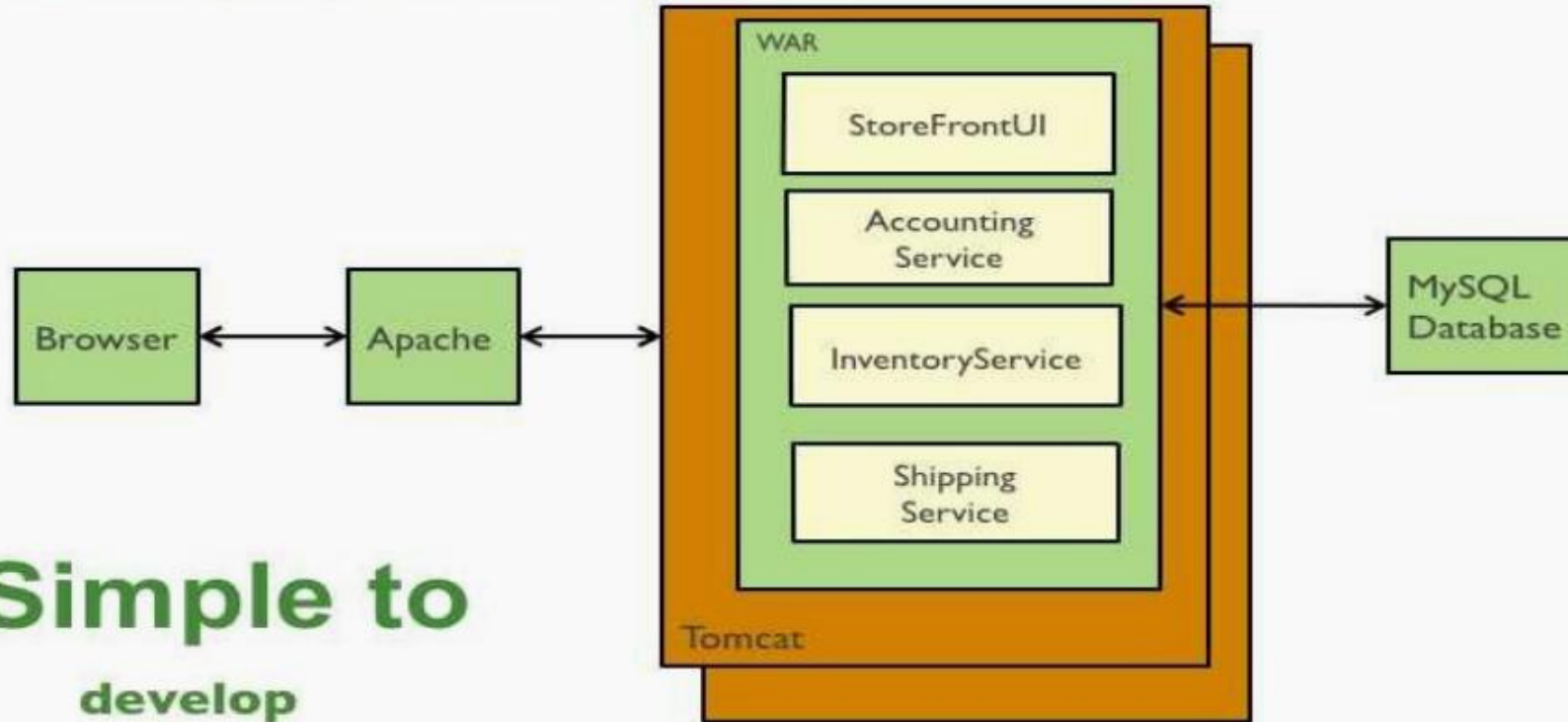
- Monolithic Architecture is the conventional application architecture in which the different functional components of an application are developed and deployed as one single application unit. It is the first architectural pattern discussed in Unit 1 of your MSA syllabus.
- The MSA book describes a typical enterprise application as having different logical components—such as UI, business logic, database access, and communication with other systems—but in a monolithic architecture these components are packaged and deployed together.

# Definition

- Monolithic Architecture is a software architecture in which an entire application is built as a single, unified codebase and deployable unit.
- In simple words:
  - All major functionalities of an application are combined into one application and are deployed together.
- For example, an e-commerce application may contain:
  - User Interface
  - Customer Management
  - Product Management
  - Order Management
  - Inventory Management
  - Payment
  - Shipping
- In a monolithic architecture, all these components are generally part of one application.

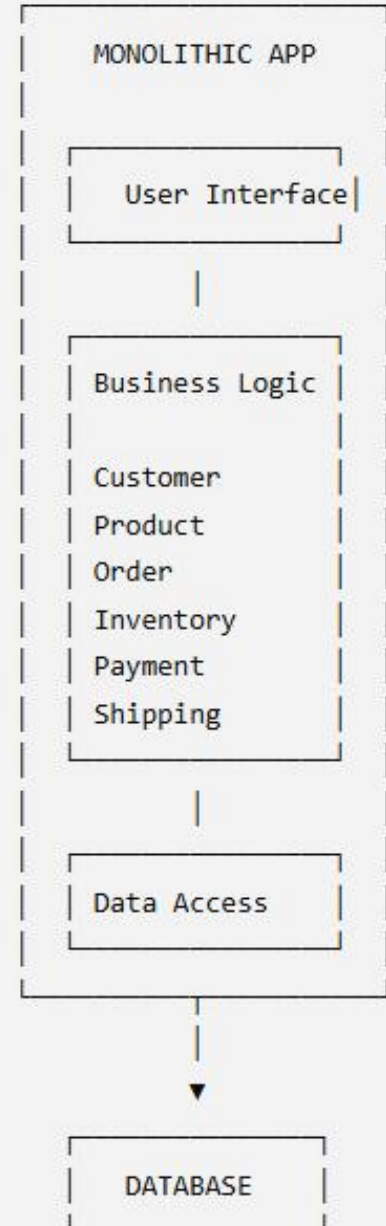
# Basic Architecture

## Traditional web application architecture



**Simple to**  
**develop**  
**test**  
**deploy**  
**scale**

CLIENT / USER



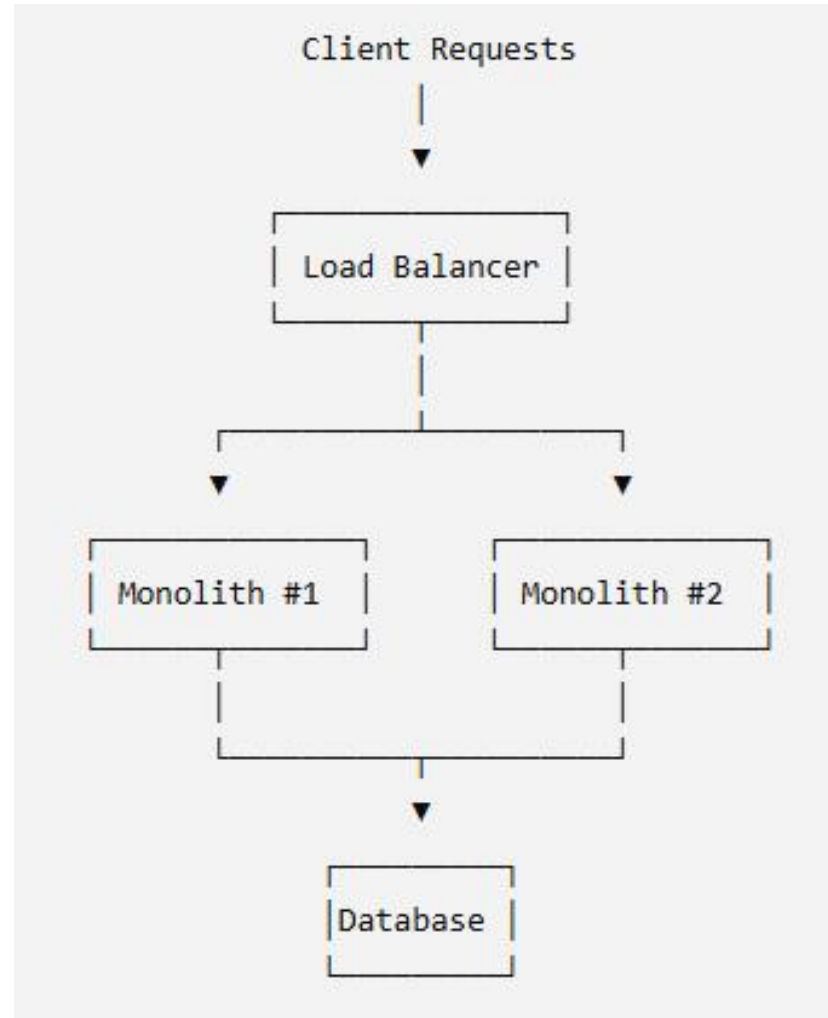
# Key idea

- Although the application may contain several logical modules, they are packaged and deployed as one application unit.
- For example, the reference material gives a Java application deployed as a single WAR file running in a web container such as Tomcat.

# Characterisitcs

- 1. Single Application Unit
- 2. Centralized Codebase
- 3. Single Deployment
- 4. Tightly Coupled Components
- 5. Common Runtime
- 6. Common Database
- 7. Scaling at Application Level

# Deployment of Monolithic Application



# Advantages

- 1. Simple to Develop
- 2. Simple to Deploy
- 3. Simple to Scale
- 4. Easy Intial Development
- 5. Easy Local Testing

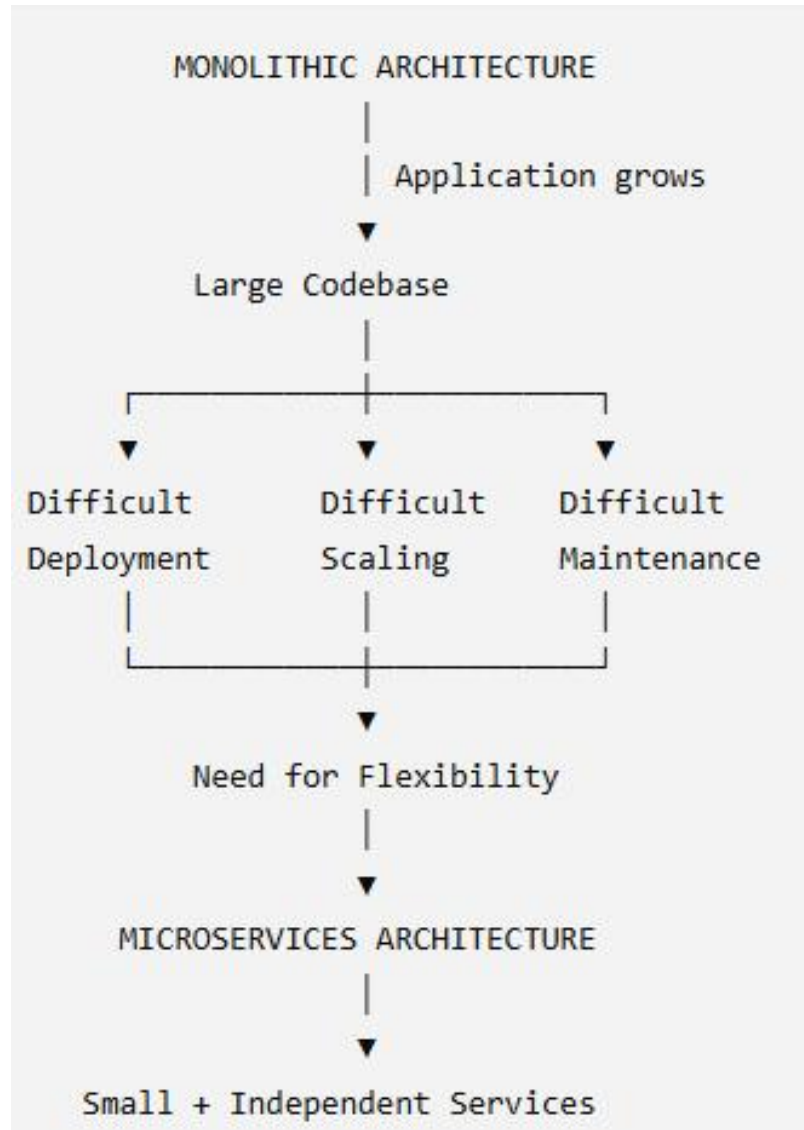
# Disadvantages

- 1. Large & Complex Codebase
- 2. Overloaded IDE
- 3. Slow Application Setup
- 4. Difficult CD
- 5. Difficult Independent Scaling
- 6. Obstacle to Scaling Development Teams
- 7. Technology Stack Dependency

# Monolithic vs MSA

| Feature              | Monolithic Architecture               | Microservices Architecture             |
|----------------------|---------------------------------------|----------------------------------------|
| Structure            | Single application                    | Collection of independent services     |
| Deployment           | Entire application                    | Services independently deployed        |
| Coupling             | Generally tighter                     | Loosely coupled                        |
| Scaling              | Entire application                    | Individual services                    |
| Database             | Common/centralized database is common | Each service can have its own database |
| Technology           | Usually common technology stack       | Different technologies can be used     |
| Failure isolation    | Lower                                 | Better                                 |
| Development teams    | More coordination required            | Autonomous teams possible              |
| Deployment frequency | More difficult                        | Easier continuous deployment           |
| Complexity           | Simpler initially                     | More distributed-system complexity     |

# What is the Need of MSA Architecture ?



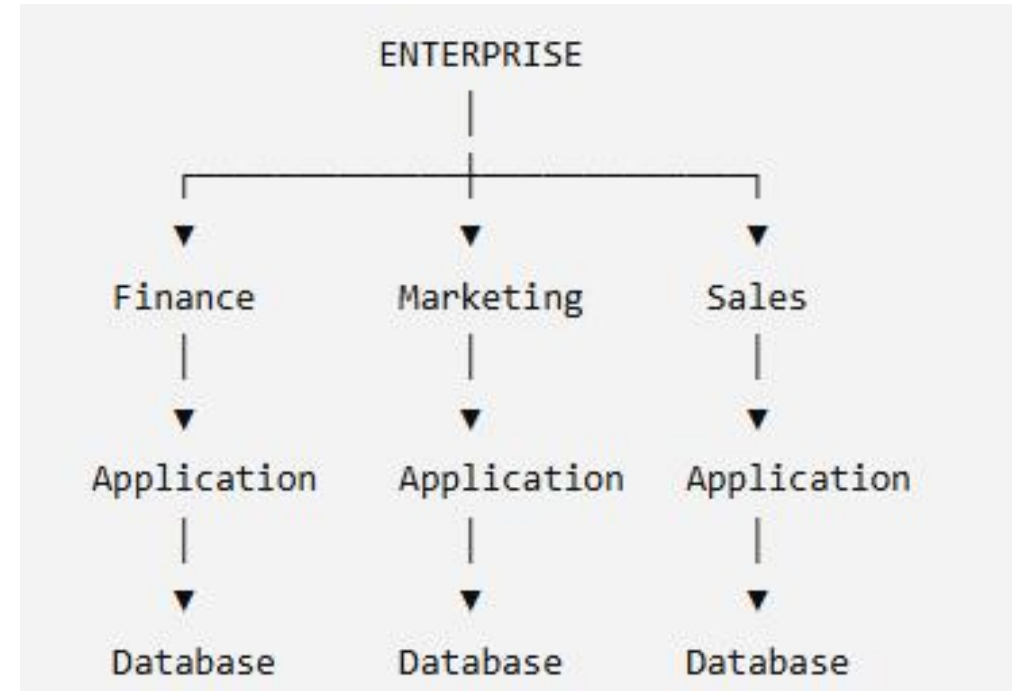
## 1.1.2 Enterprise Architecture

- Your syllabus places Enterprise Architecture (EA) immediately after Monolithic Architecture and before SOA and Microservices Architecture under “An overview of current Design Architectural patterns.”
- Enterprise Architecture (EA) is a structured approach for designing and organizing an organization's business processes, applications, data, and technology infrastructure so that they work together to achieve business objectives.
- Simple Definition :
- Enterprise Architecture is the overall blueprint of an organization that shows how its business, applications, data, and technology are organized and connected.
- The main purpose is to achieve alignment between business requirements and IT systems.

# Need of Enterprise Architecture

- Large organizations generally have:

- Many departments
- Many business processes
- Multiple software applications
- Different databases
- Different technologies
- External partners and customers



- Without proper architectural planning, these systems can become disconnected and difficult to manage.

# Main Components / Layers of Enterprise Architecture

## 1. BUSINESS ARCHITECTURE

Business goals, processes, functions

## 2. APPLICATION ARCHITECTURE

Software applications and services

## 3. DATA ARCHITECTURE

Data, databases and information flow

## 4. TECHNOLOGY ARCHITECTURE

Servers, networks, cloud, platforms

## 3.1 Business Architecture

Business Architecture describes **what the organization does**.

It includes:

- Business goals
- Business strategies
- Business processes
- Organizational structure
- Business functions
- Roles and responsibilities

### Example

For an online shopping company:

Business Functions

- |
- |— Customer Management
- |— Product Management
- |— Order Management
- |— Inventory Management
- |— Payment
- |— Delivery

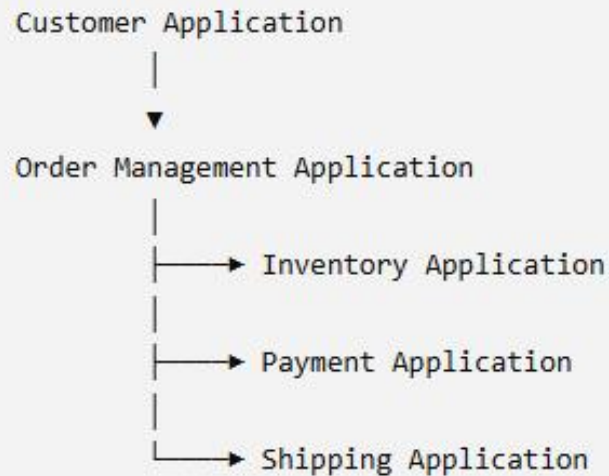


Business Architecture therefore provides the **business perspective** of the enterprise.

## 3.2 Application Architecture

Application Architecture describes the **software applications** used by the organization and how they interact.

For example:



It focuses on:

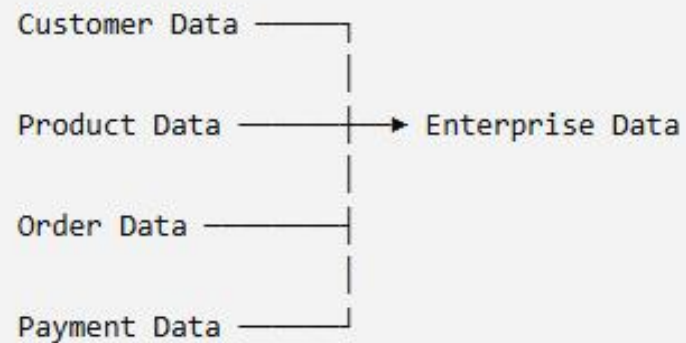
- Applications
- Application functionality
- Application dependencies
- Interfaces
- Communication between applications

### 3.3 Data Architecture

Data Architecture describes:

- What data the organization stores
- Where the data is stored
- How data is organized
- How data moves between systems
- How data is shared and managed

Example:



The goal is to ensure that important organizational information is properly managed and available to the required applications.

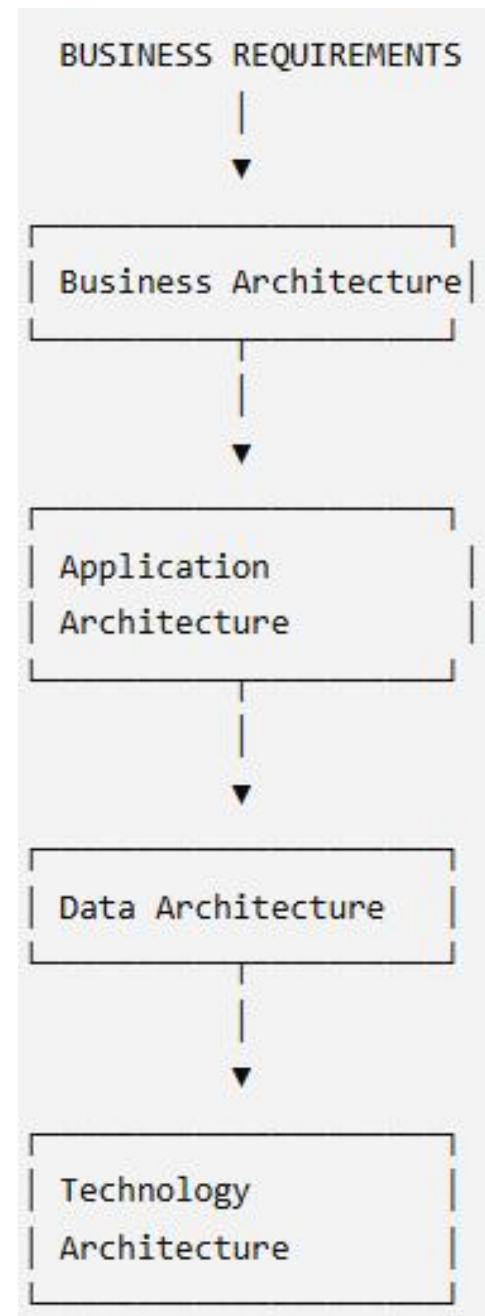
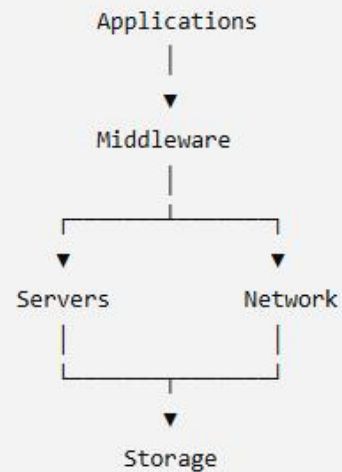
### 3.4 Technology Architecture

Technology Architecture defines the **technical infrastructure** required to run applications.

It includes:

- Servers
- Operating systems
- Networks
- Storage
- Middleware
- Cloud infrastructure
- Security infrastructure
- Computing platforms

Example:



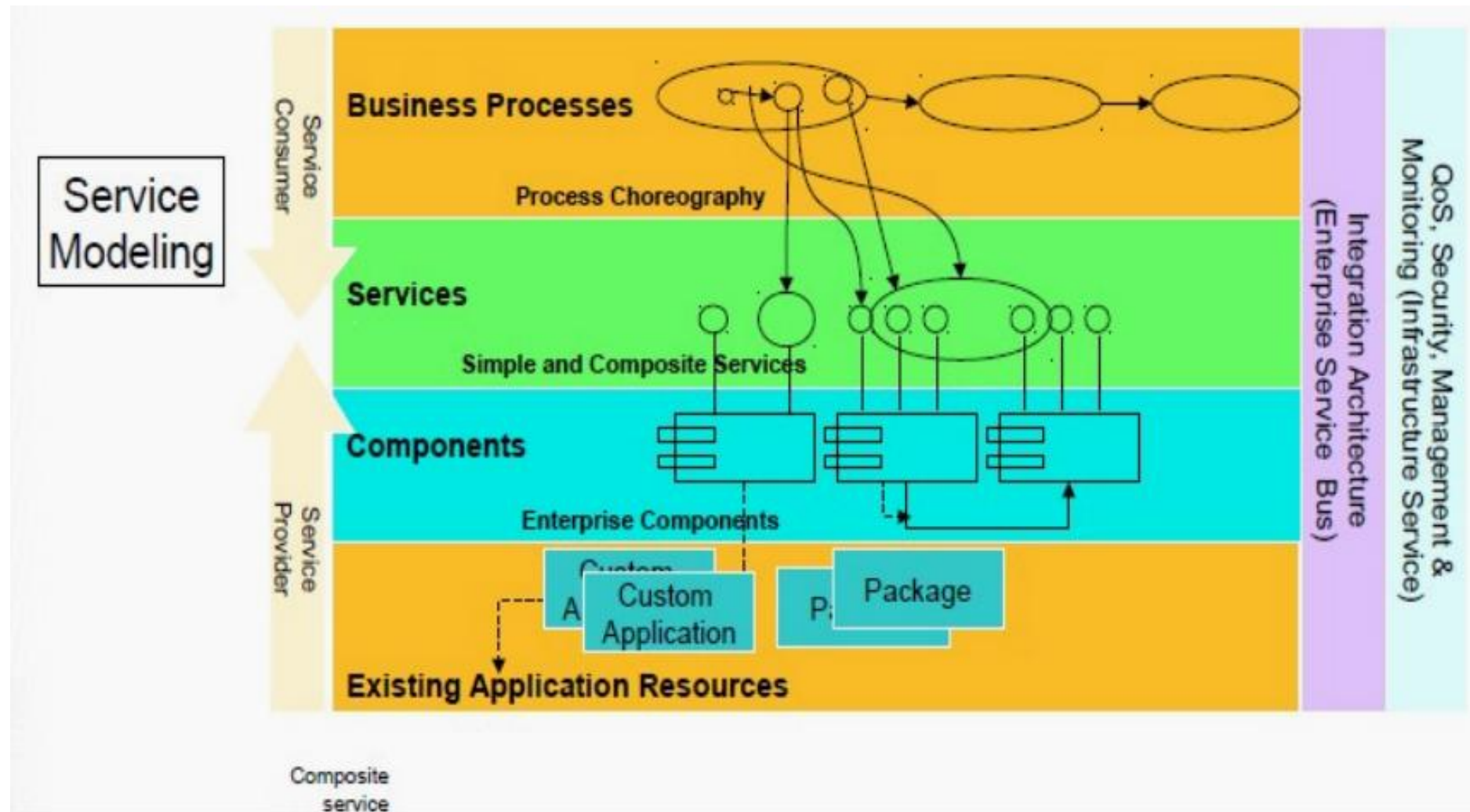
# Advantages

- 1. Better Business and IT Alignment
- 2. Improved Integration
- 3. Reduced Redundancy
- 4. Better Decision Making
- 5. Improved Resource Utilization
- 6. Better Scalability
- 7. Standardization
- 8. Easier Management of Complex IT Systems

# Limitations / Challenges

- 1. Complex Planning
- 2. Time Consuming
- 3. Cost
- 4. Resistance to Change
- 5. Difficult to Maintain

## 1.1.3 SOA - Service Oriented Architecture



# Introduction

- Service-Oriented Architecture (SOA) is an IT architectural style that supports service orientation, where business functionality is exposed as reusable and well-defined services that can be combined to support business processes.
- Simple Definition
- SOA is an architecture in which business functionality is organized as reusable, loosely coupled services that communicate through well-defined interfaces.

# Understanding SOA Through Three Important Terms

- 1. Service
- A service represents a repeatable business task.
- Examples:
  - Check customer credit
  - Open a new account
  - Process payment
  - Check inventory
  - Create an order
- A service encapsulates a functional unit of an application and exposes it through a well-defined and implementation-independent interface. Other services or client applications can consume the service.

## 2. Service Orientation

- Service orientation is a method of integrating business applications and processes as linked services with unique outcomes.
- The PPT emphasizes that service orientation is not merely a technology; it is also a way of thinking about integrating business applications and processes

# 3. Service-Oriented Architecture

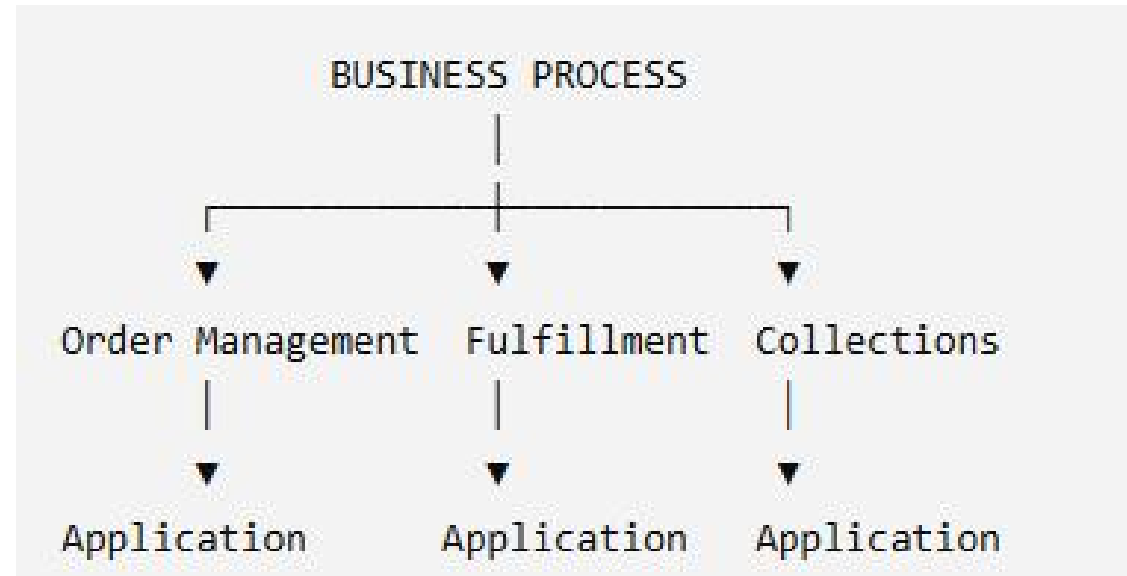
- SOA is:
- An IT architectural style that supports service orientation.
- Service
- ↓
- Repeatable Business Task
- ↓
- Service Orientation
- ↓
- Business Applications linked as Services
- ↓
- SOA
- ↓
- IT Architecture supporting this approach

# Why Was SOA Needed?

- Traditional enterprise applications often had business processes embedded inside different applications.
- Your PPT gives an Order-to-Cash example where business functions were distributed among applications such as:
  - Order Management
  - Fulfillment
  - Marketing
  - Inventory Management
  - Collections
- The problem was that business processes were embedded in separate applications, making changes difficult and creating tightly coupled business functions

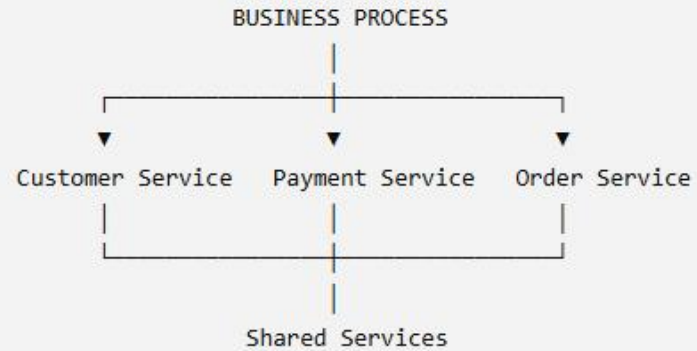
# Traditional Situation

- This causes:
  - Difficult changes
  - Long development cycles
  - Tight coupling
  - Difficult management
  - Difficult reuse
- 
- The PPT summarizes the result as:
  - Business cannot respond quickly to changing demand.



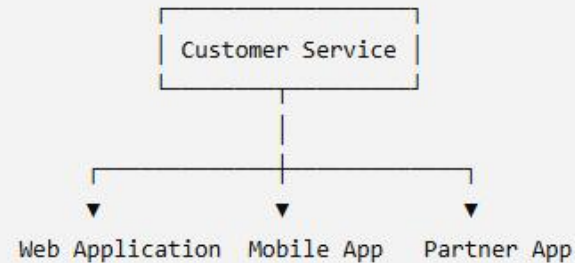
# 4. SOA Approach

SOA attempts to organize business functionality as **reusable services**.



Different applications or business processes can consume these services.

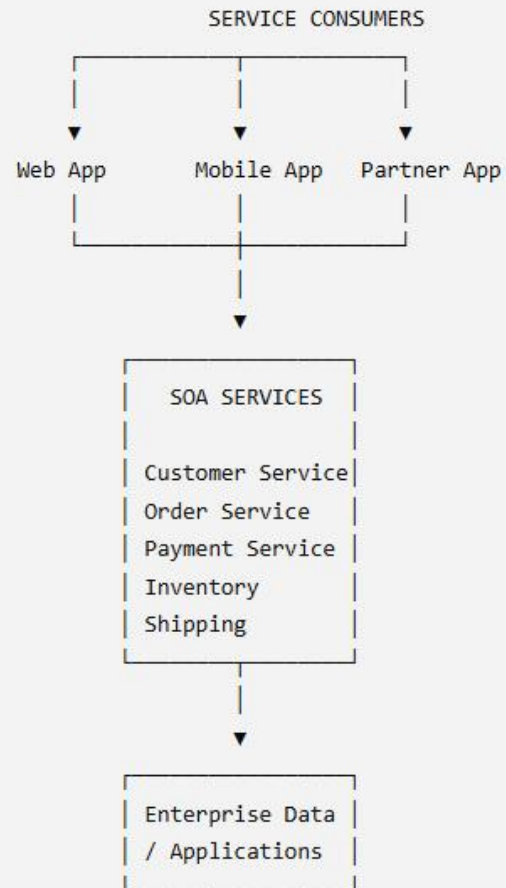
For example:



The same business capability can therefore be reused by multiple consumers.

## 5. SOA Architecture

A simplified SOA architecture can be represented as:



The important concept is that consumers interact with **services through defined interfaces**, rather than directly depending on their internal implementation.

## 6. Characteristics of SOA

- 1. Reusable Service
- 2. Loose Coupling
- 3. Well-Defined Interface
- 4. Interoperability
- 5. Reusability Across Enterprise
- 6. Composability

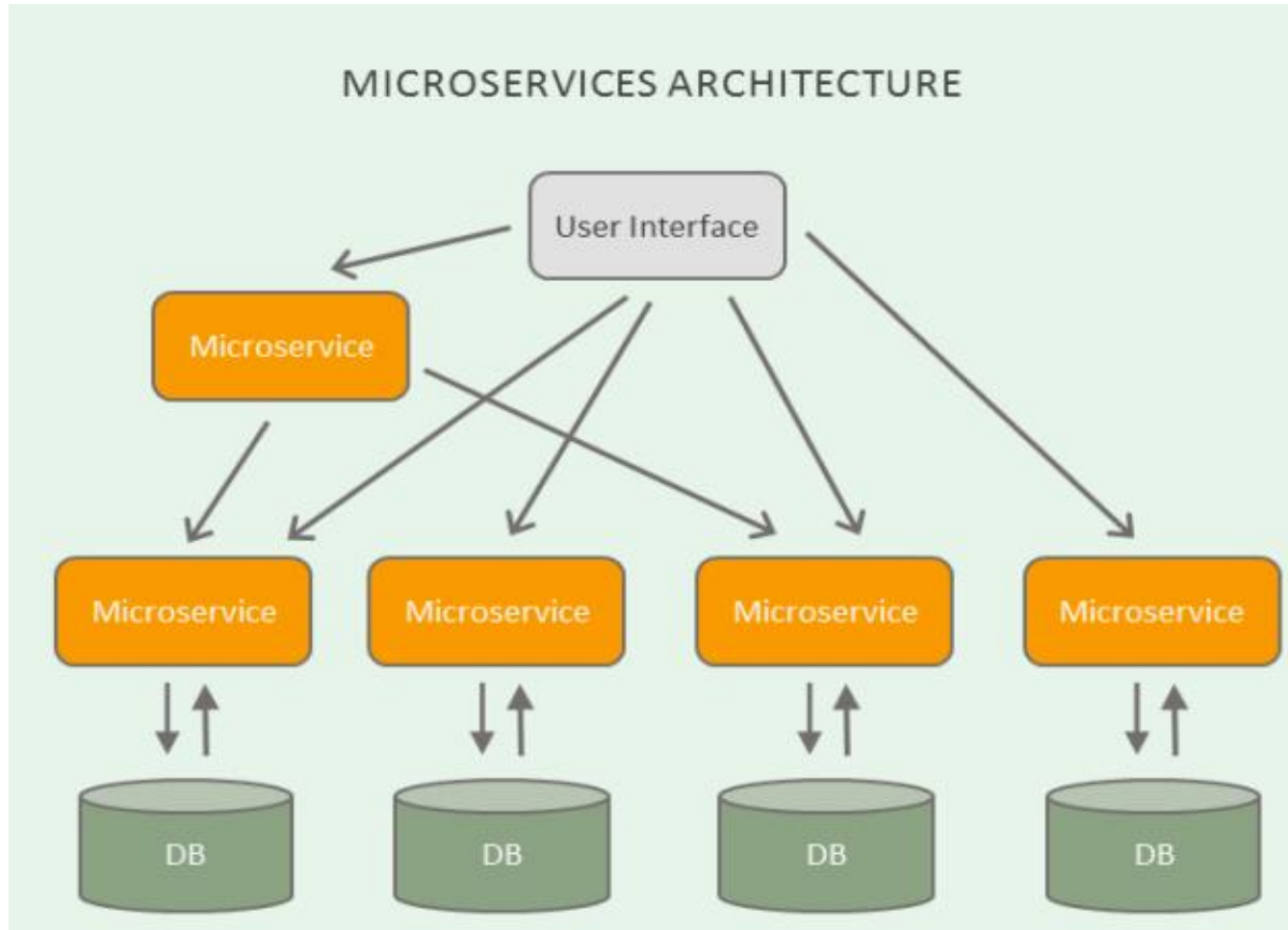
# Disadvantages of SoA

- 1. Increased Complexity
- 2. Network Dependency
- 3. Service Management
- 4. Security Challenges
- 5. Performance Overhead
- 6. Governance Requirement

## 1.1.4 Micro Services Architecture

- Microservices Architecture (MSA) is the fourth and most important architectural pattern in Unit 1 of your syllabus. The syllabus places it after Monolithic, Enterprise, and Service-Oriented Architecture and then moves into Designing MSA through decomposition, business capability, subdomains, service-per-team, and DDD.
- Your MSA PPT defines the architecture around small, focused, loosely coupled services that can be independently deployed and updated.

# Diagram

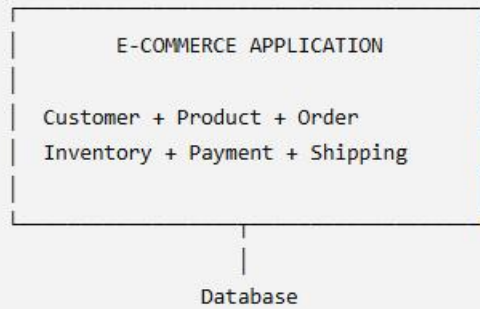


# Definition

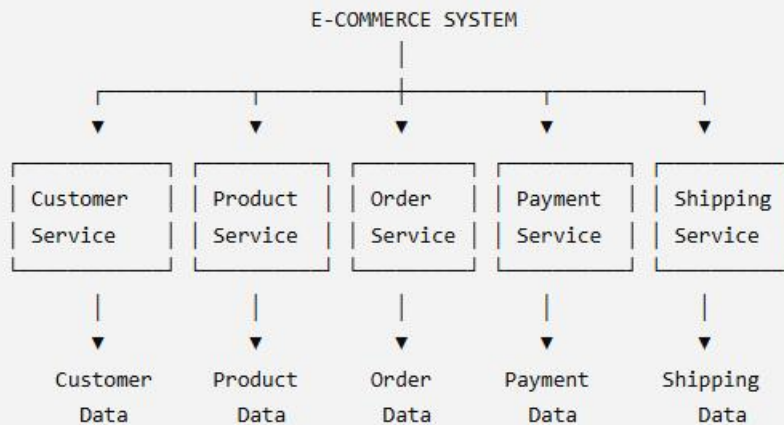
- Microservices Architecture is an architectural style in which an application is structured as a collection of small, loosely coupled, independently deployable services, where each service focuses on a specific business capability or responsibility.
- The reference book describes the solution as structuring an application as a set of loosely coupled, collaborating services. Each service should be maintainable, testable, independently deployable, and capable of being developed by a small team.
- Simple Definition :
- Microservices Architecture divides a large application into small, independent services, with each service responsible for a specific business function.

# Basic Idea of MicroServices

- Consider an E-Commerce Application.



In Microservices Architecture:



# What is a Microservice?

- A microservice is a small, independently deployable application/service that performs a specific business responsibility.
  - Each microservice is a miniature application.
  - Each focuses on one task/business capability.
  - Each can be deployed and updated independently.
  - Services are loosely coupled.
  - Each has a well-defined interface, such as REST APIs.
  - Services should be lightweight for smaller memory footprints and faster startup.

# Characteristics

## 4.1 Small and Focused Services

Each service should implement a **small set of strongly related functions**.

For example:

Order Service → Order-related operations

rather than:

Order + Payment + Inventory + Customer + Shipping

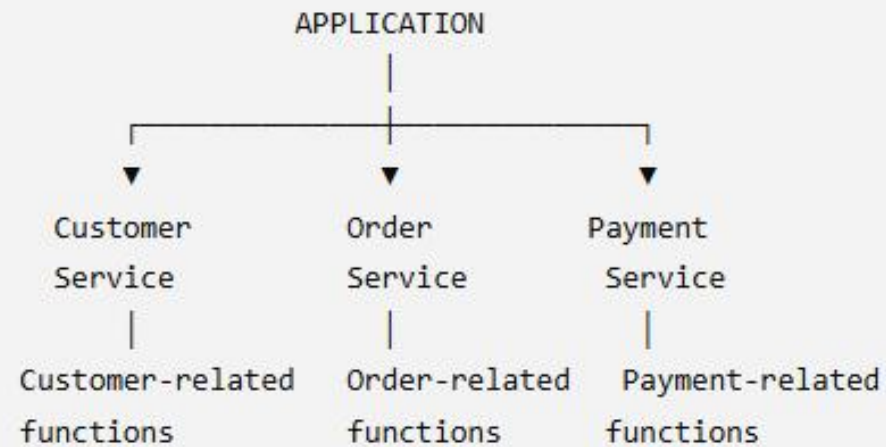
The reference book calls for services to be **cohesive**, meaning that the functions within a service should be strongly related.  msabook

# Characteristics

## 4.2 Single Business Responsibility

A microservice should focus on a specific business responsibility.

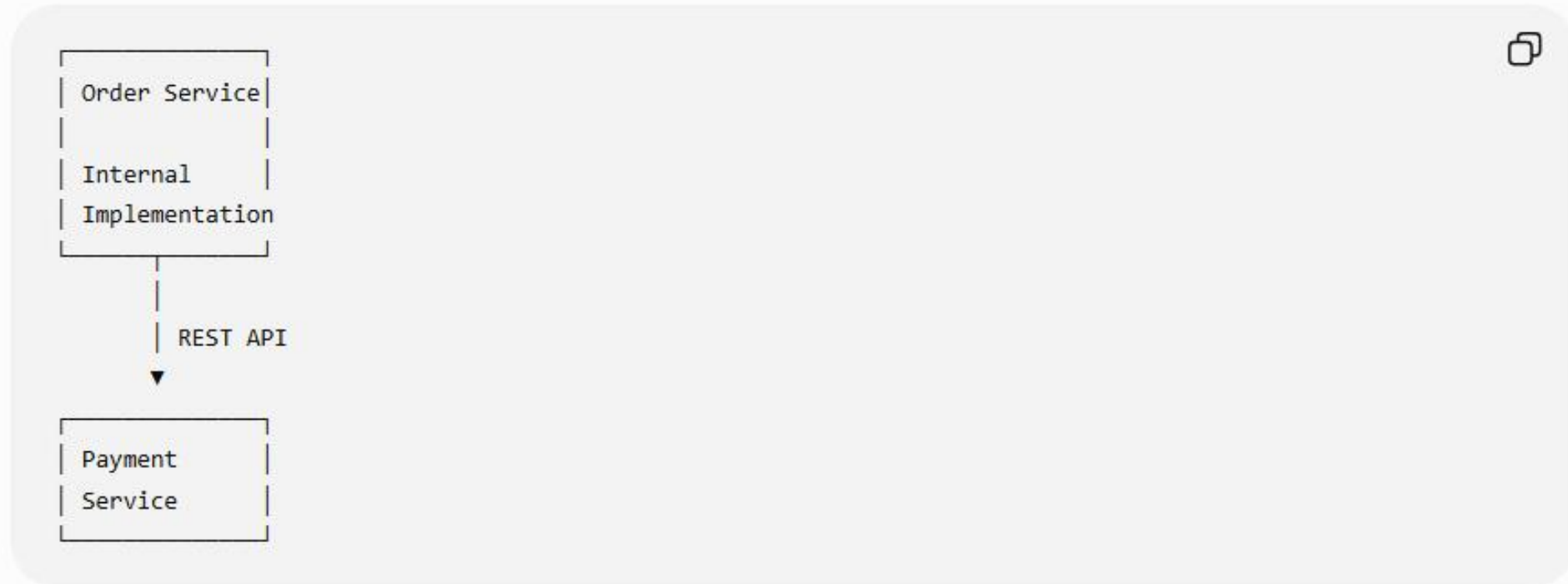
This follows the idea of the **Single Responsibility Principle (SRP)**.



# Loose Coupling

**Loose coupling** means that services have minimal dependency on each other's internal implementation.

A service exposes an interface/API:

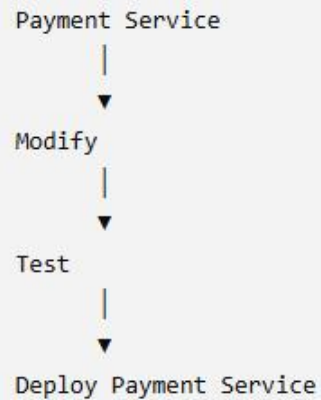


The consumer interacts through the **API**, rather than directly accessing the internal implementation.

# Independent Deployment

One of the most important features of MSA is **independent deployment**.

Suppose we modify only the Payment Service.



We do **not** need to redeploy the entire application.

This enables:

- Faster releases
- Continuous delivery
- Independent development
- Lower deployment risk

# Independent Scaling

- Different services can have different resource requirements.
- For example:
  - Search Service → Very High Traffic
  - Payment Service → Medium Traffic
  - Shipping Service → Low Traffic
- With microservices:
- Search Service
  - └─ Instance 1
  - └─ Instance 2
  - └─ Instance 3
  - └─ Instance 4
- Only the required service can be scaled.
- This is an important improvement over monolithic architecture, where the entire application generally has to be scaled together.

# Technology Independence / Polyglot Development

- Microservices allow different services to use technologies best suited to their requirements.
- For example:
  - Customer Service → Java
  - Order Service → .NET
  - Payment Service → Python
  - Search Service → Node.js,

# Independent Data Management

- Conceptually:

- Customer Service —▶ Customer DB
- Order Service —▶ Order DB
- Payment Service —▶ Payment DB
- Inventory Service —▶ Inventory DB

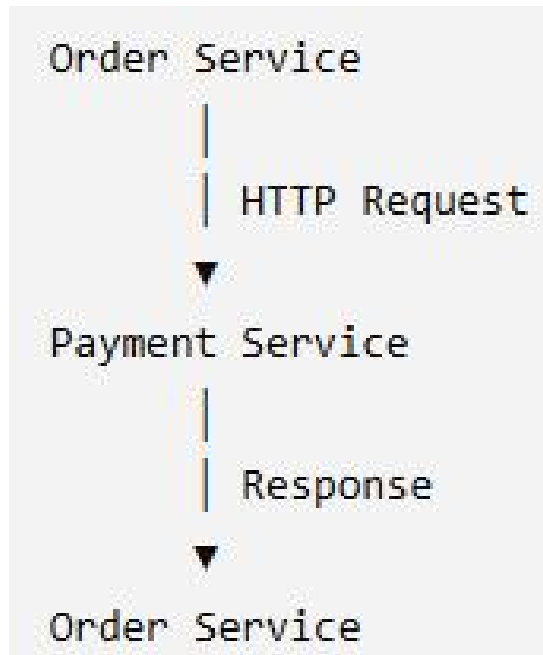
- This is commonly called the Database per Service pattern.

- Important:

- A shared database can create strong coupling between services. Therefore, independent data ownership is preferred when applying microservices principles.

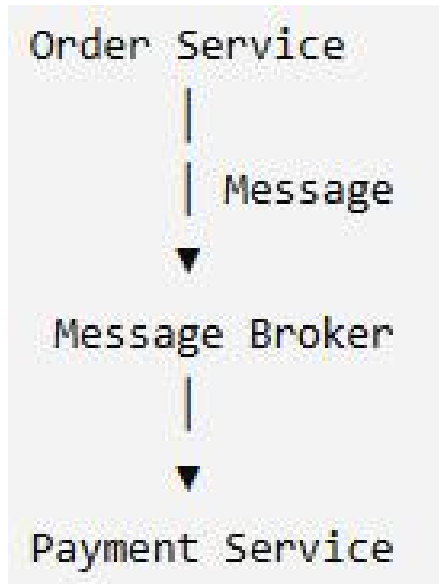
# Communication Between Services

- Microservices need to communicate with each other.
- The reference book identifies two broad communication approaches.
- **A. Synchronous Communication**
- Example:

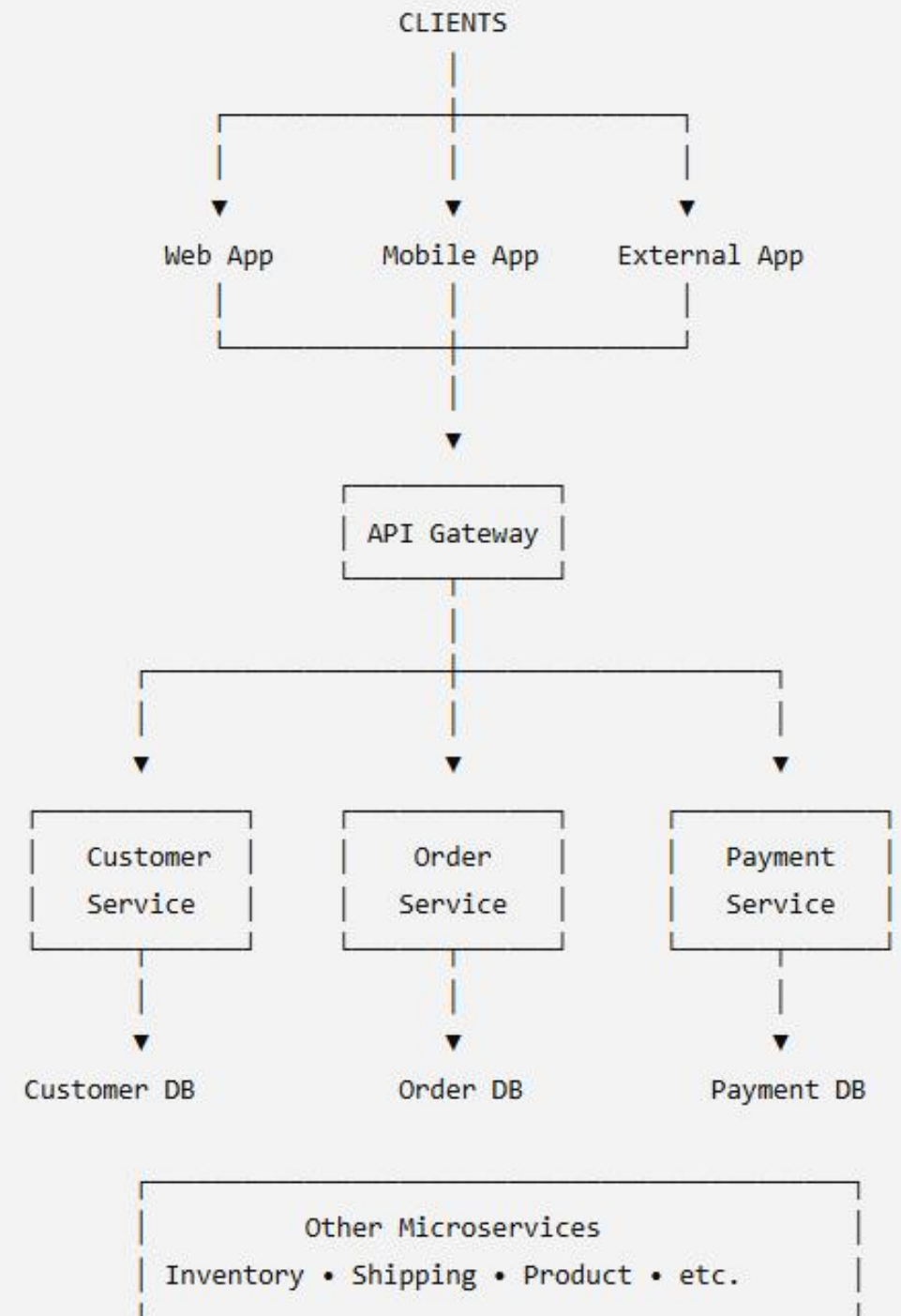


# Communication Between Services

- B. Asynchronous Communication



# Microservices Architecture : Overall Diagram



# Benifits

- 1. Better Maintainbility
- 2. Better Testability
- 3. Independent Deployment
- 4. Autonomous Teams
- 5. Technology Freedom
- 6. Fault Isolation
- 7. CD - Continuous Delivery

# Important MSA Service Qualities

- 1. Lightweight
- 2. Externalized
- 3. Consistent
- 4. Resilient
- 5. Good Citizens
- 6. Versioned

# Challenges / Disadvantages of Microservices

- 1. Distributed System Complexity
- 2. Network Communication
- 3. Data Consistency
- 4. Monitoring Difficulty
- 5. Deployment Management
- 6. Service Discovery
- 7. Security
- 8. Increased Infrastructure Requirements

## 16. Microservices and Service per Team

Microservices also influence how development teams are organized.

The reference book states that a high-performance organization can consist of small, long-lived, autonomous and cross-functional teams, and connects this with **Conway's Law**.  msabook

A typical model:

```
graph LR; A[Team A] --> CS[Customer Service]; B[Team B] --> OS[Order Service]; C[Team C] --> PS[Payment Service]; D[Team D] --> IS[Inventory Service];
```

Team A —→ Customer Service  
Team B —→ Order Service  
Team C —→ Payment Service  
Team D —→ Inventory Service

Each team can:

- Develop
- Test
- Deploy
- Maintain
- Scale

its own service.

# Real World Example

- The reference book mentions large-scale websites that evolved from monolithic architectures toward service-oriented/microservice-style architectures, including Netflix, Amazon and eBay.
- For examination purposes, you can mention these as examples of large-scale systems that adopted service-oriented/microservice approaches.

# Comparision Between Various Architecture

| <u>Feature</u>                 | <u>Monolithic</u>                  | <u>Microservices</u>                        |
|--------------------------------|------------------------------------|---------------------------------------------|
| <b>Architecture</b>            | Single application                 | Multiple services                           |
| <b>Codebase</b>                | Usually one large codebase         | Multiple smaller codebases                  |
| <b>Deployment</b>              | Whole application                  | Individual services                         |
| <b>Scaling</b>                 | Whole application                  | Individual services                         |
| <b>Coupling</b>                | Generally tighter                  | Loosely coupled                             |
| <b>Business responsibility</b> | Multiple responsibilities together | Focused responsibility per service          |
| <b>Database</b>                | Common database is common          | Independent database ownership is preferred |
| <b>Technology</b>              | Common technology stack            | Technology can vary by service              |
| <b>Team structure</b>          | More coordination                  | Autonomous teams                            |
| <b>Testing</b>                 | Large application testing          | Smaller service-level testing               |
| <b>Failure isolation</b>       | Lower                              | Better isolation                            |
| <b>Development</b>             | Becomes difficult at large scale   | Supports parallel development               |
| <b>Complexity</b>              | Lower initially                    | Higher distributed-system complexity        |

# Microservices vs SOA

## SOA

Focuses strongly on enterprise service integration and reuse

Services may be comparatively larger

Often has stronger centralized governance

Enterprise-wide integration is a major concern

Shared infrastructure can be common

Can use centralized enterprise mechanisms

## Microservices

Focuses strongly on small, independently deployable services

Services are generally smaller

Greater team/service autonomy

Application agility and independent deployment are major concerns

Independent service ownership is emphasized

Lightweight APIs and messaging are common

# Important MSA Design Principles

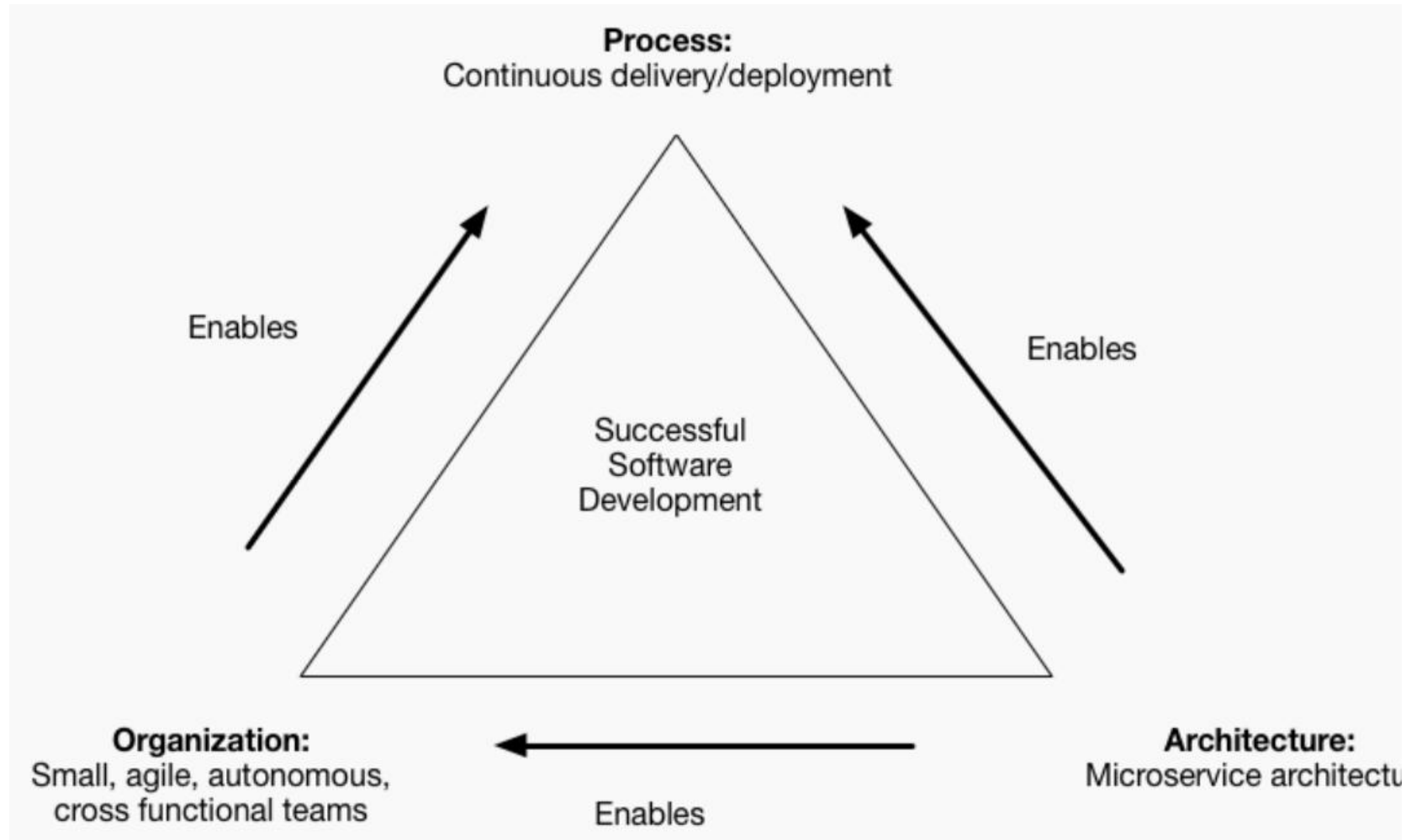
IMP.

- **1. Stable Architecture**
  - The service boundaries should remain reasonably stable.
- **2. Cohesive Services**
  - Each service should contain strongly related functions.
- **3. Common Closure Principle**
  - Things that change together should be packaged together.
  - The objective is to ensure that a change affects as few services as possible.
- **4. Loose Coupling**
  - Services communicate through APIs/interfaces and hide their internal implementation.
- **5. Testability**
  - Each service should be easy to test.
- **6. Small Team Size**
  - The service should be manageable by a small team.
  - The reference book gives the two-pizza team idea, roughly 6–10 people in one section.
- **7. Team Autonomy**
  - Teams should be able to develop and deploy their services with minimal dependence on other teams.

## 1.2.1 Decomposition of Monoliths

- Decomposition of Monoliths is the process of breaking a large monolithic application into smaller, independent and loosely coupled services that can be developed, tested, deployed, and scaled separately.
- Why Decompose a Monolith?
- A large monolithic application may become:
  - Difficult to maintain
  - Difficult to test
  - Difficult to deploy
  - Difficult to scale
  - Highly dependent internally
- Decomposition helps overcome these limitations and prepares the application for Microservices Architecture.

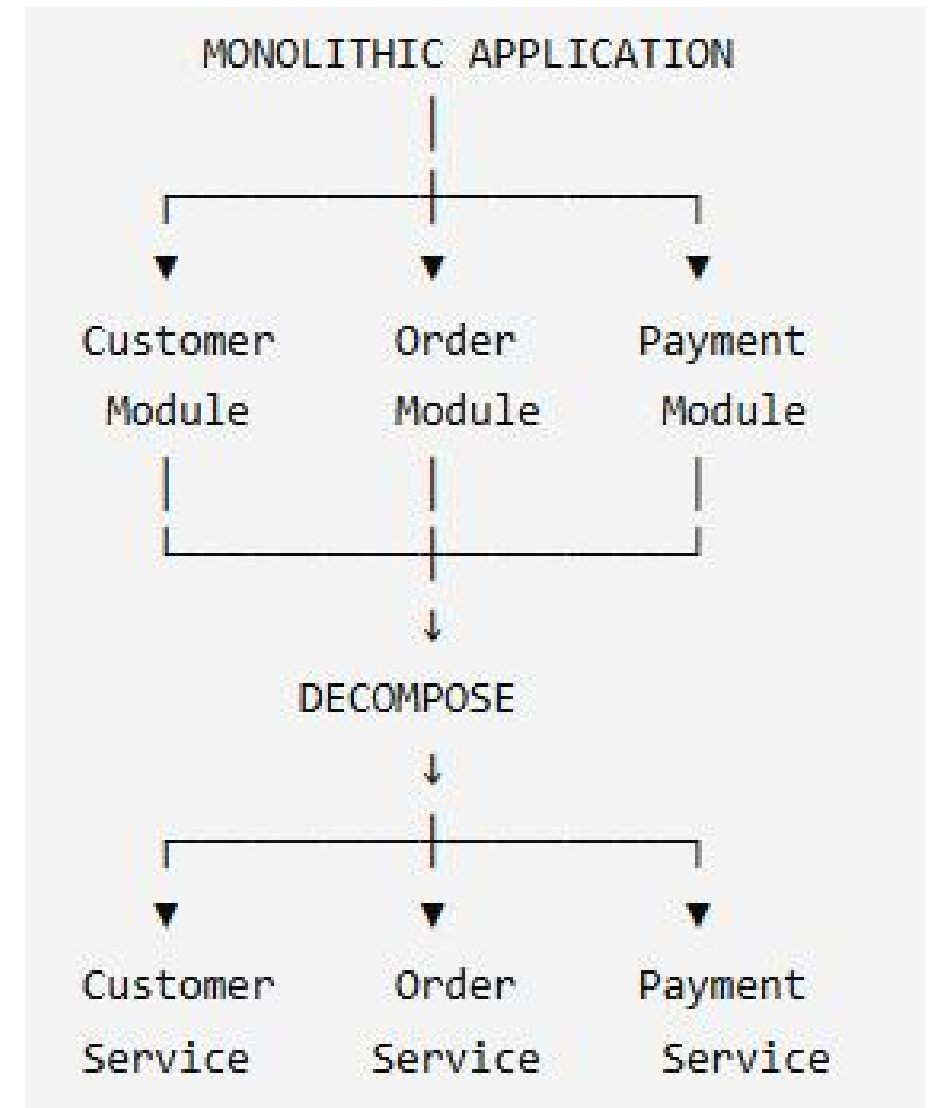
# Diagram



# Basic Concept

## Principles of Good Decomposition

1. Cohesion – Related functionality should remain together.
2. Loose Coupling – Minimize dependencies between services.
3. Single Responsibility – Each service should have a clear responsibility.
4. Independent Deployment – Services should be deployable independently.
5. Testability – Each service should be easy to test.
6. Team Autonomy – A small team should be able to own and manage a service.



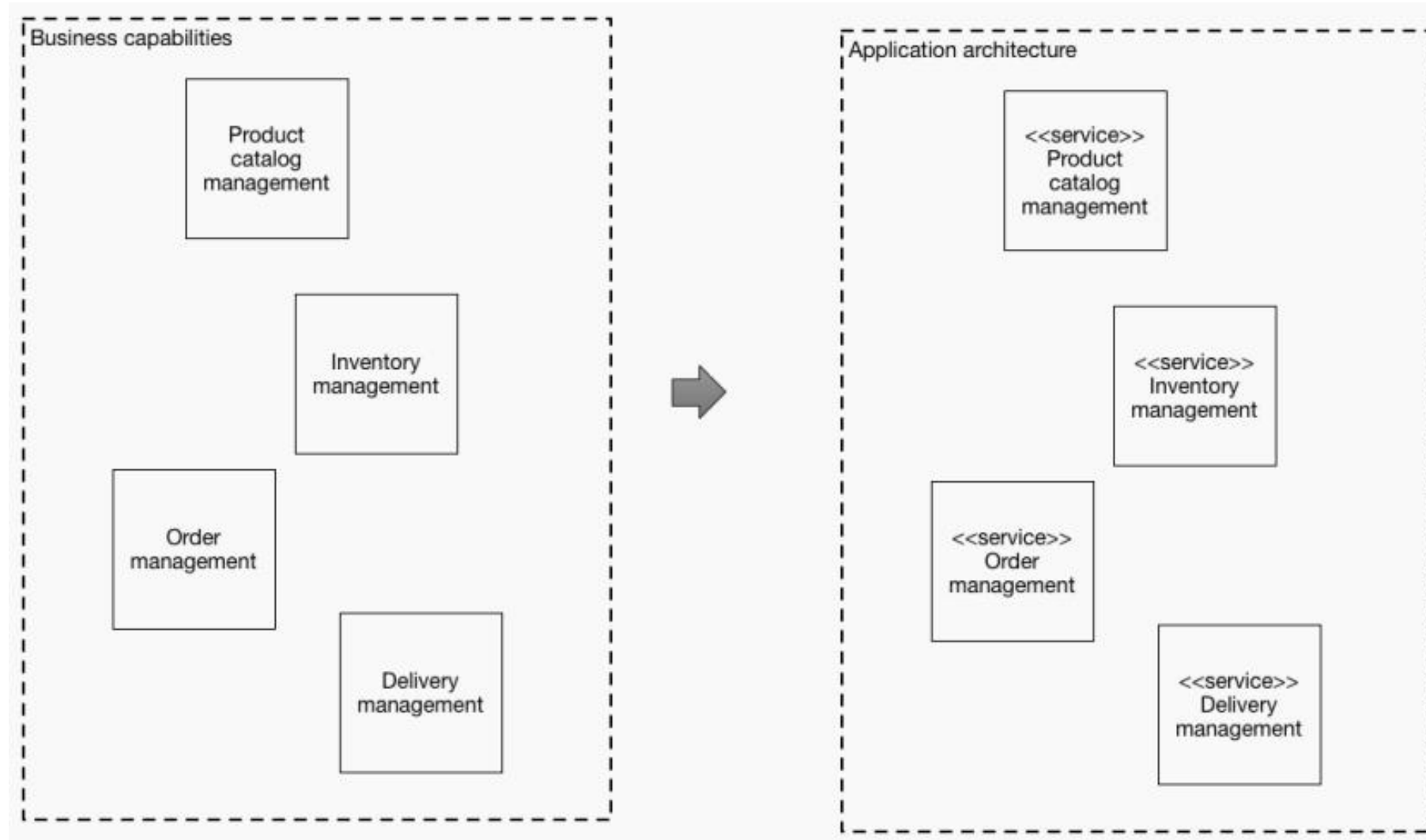
# Main Decomposition Approaches

- According to your syllabus, the following approaches are studied:
- Decompose by Business Capability
- Decompose by Subdomain
- Service per Team
- Domain-Driven Design (DDD)

## 1.2.2 Decompose by Business Capability

- Decomposition by Business Capability is a method of designing microservices by identifying the main business capabilities/functions of an organization and creating a service around each capability.
- Definition : A business capability represents what a business does to generate value. Each major business capability can be implemented as a separate microservice.

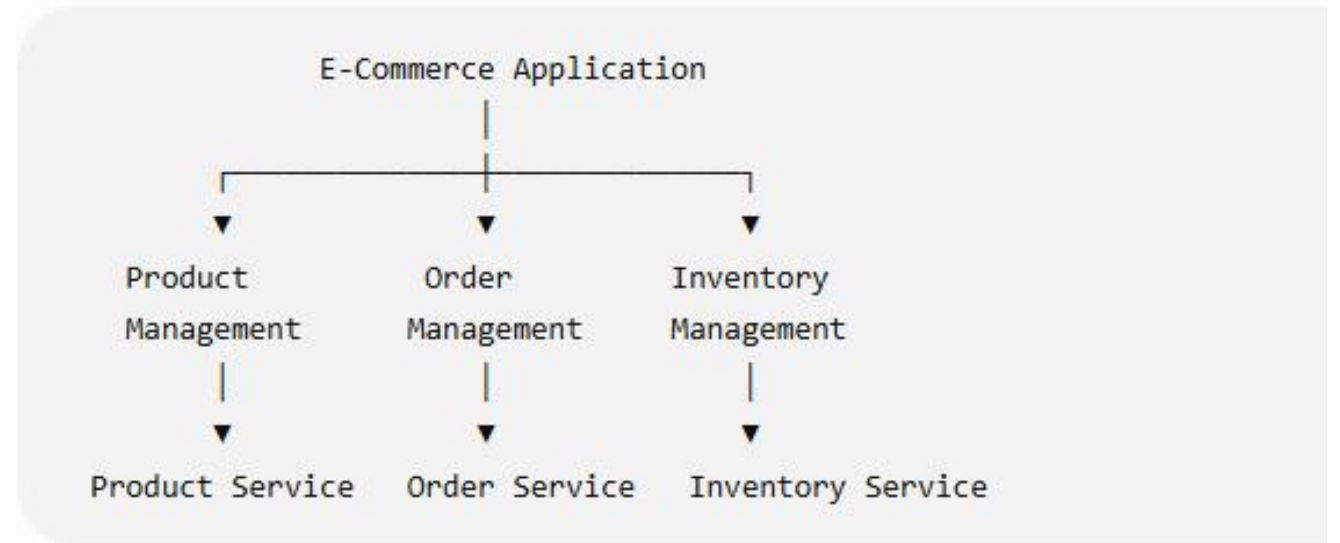
# Diagram



# Example

- ADVANTAGES
- Services have clear business responsibilities.
- Reduces coupling between business functions.
- Supports independent development and deployment.
- Makes services easier to understand and maintain.
- Aligns microservices with actual business operation

## Example – E-Commerce System

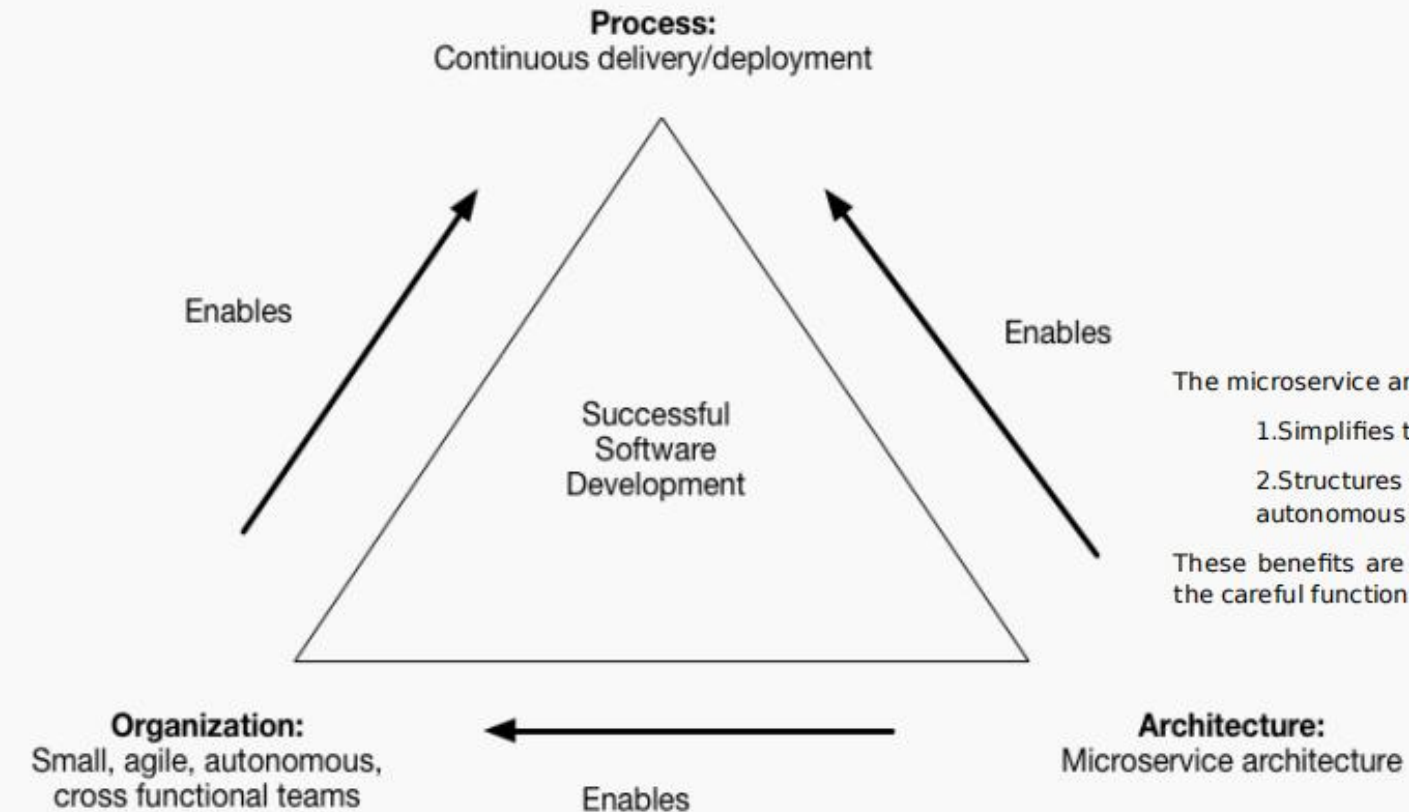


Other possible capabilities include:

- Customer Management → **Customer Service**
- Payment Processing → **Payment Service**
- Shipping → **Shipping Service**

## 1.2.3 Decompose by SubDomain

You are developing a large, complex application and want to use the microservice architecture. The microservice architecture structures an application as a set of loosely coupled services. The goal of the microservice architecture is to accelerate software development by enabling continuous delivery/deployment.



The microservice architecture does this in two ways:

1. Simplifies testing and enables components to be deployed independently
2. Structures the engineering organization as a collection of small (6-10 members), autonomous teams, each of which is responsible for one or more services

These benefits are not automatically guaranteed. Instead, they can only be achieved by the careful functional decomposition of the application into services.

# Key Idea

- Instead of dividing the application based only on technical modules, we identify business/domain boundaries.
  - Business Domain
    - ↓
  - Identify Subdomains
    - ↓
  - Define Boundaries
    - ↓
  - Create Microservices
- **Advantages**
    - Services are aligned with business domains.
    - Creates clear service boundaries.
    - Reduces coupling between different business areas.
    - Makes services easier to maintain and evolve.
    - Supports independent development by teams.

## 1.2.4 Service Per Team

- Service per Team is a microservices design approach in which a small, autonomous development team owns and manages one or more microservices.
- Definition : Service per Team means assigning ownership of a microservice to a dedicated team that is responsible for its development, testing, deployment, and maintenance.

# Basic Concept

## Key Idea

The team should have enough autonomy to make decisions and deploy its service without requiring continuous coordination with other teams

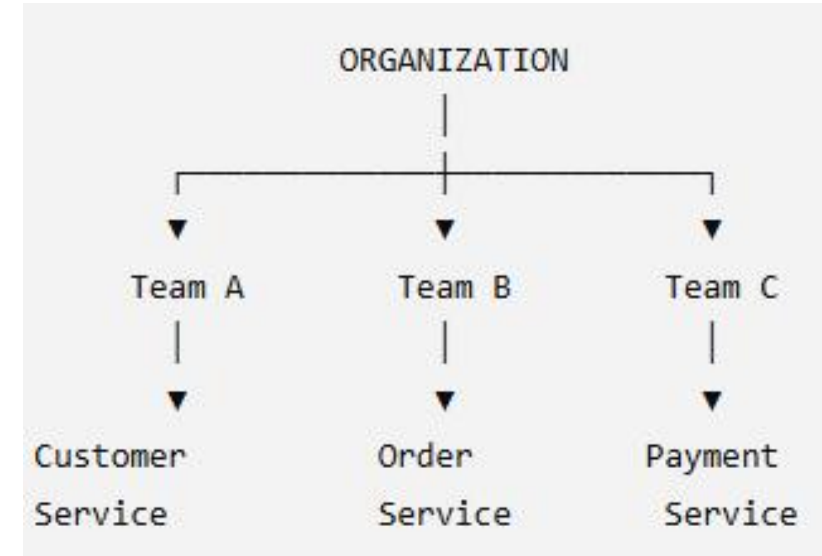
Team



Develop → Test → Deploy → Maintain



Own the Service



Each team has responsibility for its service throughout its lifecycle.

## Responsibilities of the Team

A team can be responsible for:

- Development
- Testing
- Deployment
- Maintenance
- Monitoring
- Improvement of the service

# Advantages

- Team autonomy – Teams can work independently.
- Faster development – Less coordination between teams.
- Clear ownership – Every service has an accountable team.
- Independent deployment – Teams can release their service independently.
- Better maintainability – Teams understand their own service deeply.

# DDD - Domain Driven Design

MSA

# Introduction

- Domain-Driven Design (DDD) is an important approach for designing Microservices Architecture. It focuses on understanding the business domain and connecting the software implementation with the core business concepts.
- Domain-Driven Design is a software development approach that focuses on solving complex business problems by designing the software around the business domain and its core concepts.
- In DDD, developers and business/domain experts work together using a common terminology to understand the business and build an appropriate application model.

# Simple Idea

Business Domain



Understand Business



Identify Subdomains



Define Bounded Contexts



Create Domain Model



Design Microservices

# Importance of DDD in MSA.

- One of the biggest challenges in Microservices Architecture is:
- Where should we create the service boundaries?
- DDD helps answer this question by analyzing the business domain rather than simply dividing technical components.
- Complex Business
  - ↓
  - DDD
    - ↓
    - Understand Business Areas
      - ↓
      - Define Clear Boundaries
        - ↓
        - Microservices

# Important Terms of DDD

- Domain
- Subdomains
- Domain Model
- Bounded Context
- Ubiquitous Language
- Context Map
  - It also identifies Entity, Value Object, Aggregate, Service, Factory, and Repository as important tactical building blocks of DDD,

# Domain

- A Domain represents the overall business/problem area that the software is intended to solve.
- For example:

E-Commerce System



E-Commerce

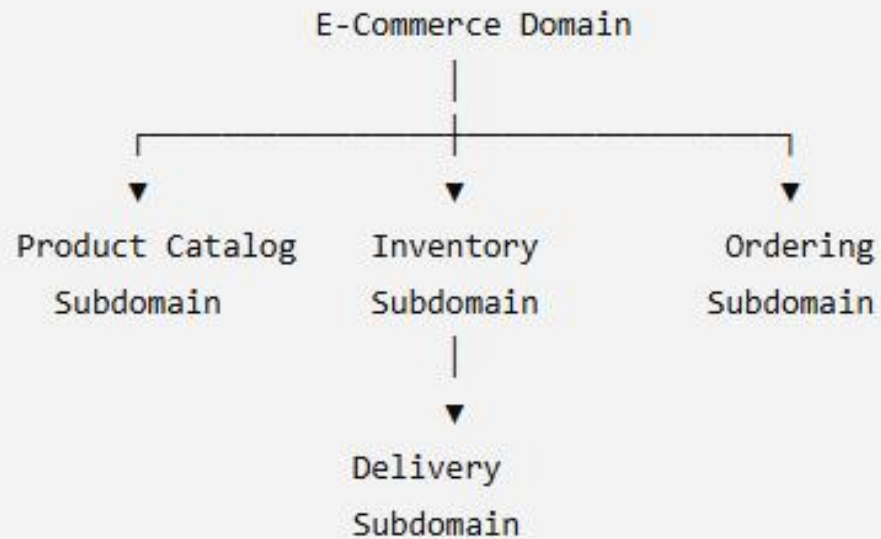
Domain

- The domain can contain several different business areas.
- According to the reference book:
- The application's problem space — the business — is called the domain.

# Subdomain

- A large domain can be divided into smaller subdomains, where each subdomain represents a different part of the business.

## Example: Online Store



# Types of Subdomains

- 1 Core Subdomain
- The Core Subdomain is the most valuable part of the business and provides its major competitive differentiation.
- Core
- ↓
- Key business differentiator
- ↓
- Highest business value

# Types of SubDomain

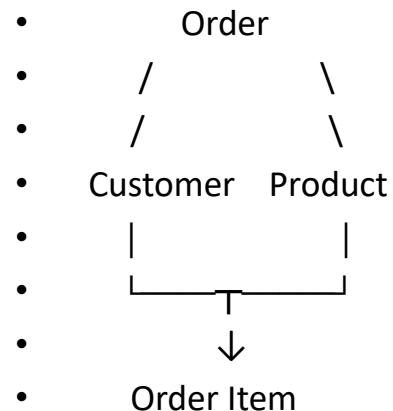
- 2. Supporting Subdomain
  - A Supporting Subdomain supports the business but is not a major differentiator.
  - It can be implemented internally or potentially outsourced.
- 3 Generic Subdomain
  - A Generic Subdomain is not specific to the business and can often be implemented using off-the-shelf software.

## Quick Comparison

| Subdomain  | Meaning                                                            |
|------------|--------------------------------------------------------------------|
| Core       | Main business differentiator                                       |
| Supporting | Supports business but is not a differentiator                      |
| Generic    | Common functionality that can use existing/off-the-shelf solutions |

# Domain Model

- A Domain Model represents the important business concepts, relationships, and rules of a particular domain or subdomain.
- It provides a model that connects the business requirements with the software implementation.
- Example
  - For an Order domain:



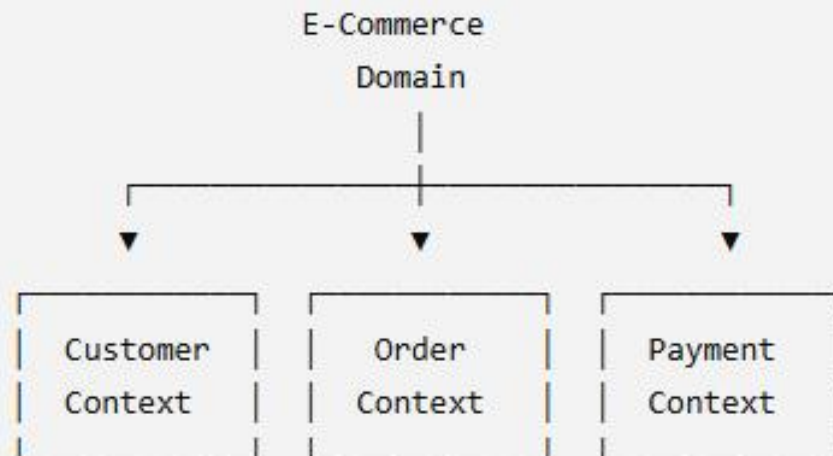
# Bounded Context

- Bounded Context is one of the most important DDD concepts for Microservices Architecture.
- A bounded context defines a clear boundary within which a particular domain model and its terminology have a specific meaning.

Domain Model + Database Schema + UI +  
Web Services/API

# Example & Importance

## Example



## Why Important?

Bounded contexts help define clear boundaries for microservices.

Bounded Context



Service Boundary



Microservice

# Ubiquitous Language

- Ubiquitous Language is a common language shared by developers and business/domain experts.
- The same business terms should be used consistently in:
  - Discussions
  - Requirements
  - Domain model
  - Code
  - Documentation

# Example

Suppose the business uses the term “Order.”

- Developers, testers and business experts should all use Order rather than having different technical terms for the same concept.

Business Expert

|

| Common Language



Developer

|

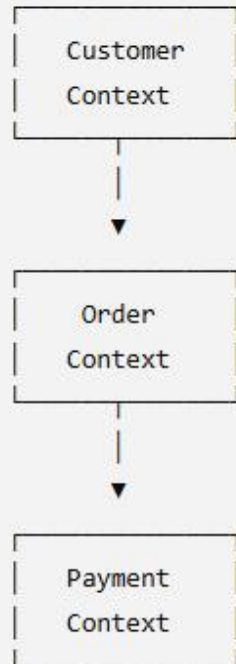


Code

# Context Map

A **Context Map** describes how different bounded contexts **relate and interact with each other**.

## Example



It provides a high-level view of relationships between different parts of the domain.

# Tactical Building Blocks of DDD

- 1. Entity
- An Entity is an object that has a distinct identity.
- Example:
  - Customer
  - Customer ID = 101
- Even if other properties change, the customer's identity remains.

# Tactical Building Blocks of DDD

- 2. Value Object
- A Value Object represents an object whose importance is based on its value rather than identity.
- Examples:
  - Address
  - Money
  - Date range

# Tactical Building Blocks of DDD

- 3. Aggregate
  - An Aggregate is a group of related domain objects treated as a single unit for consistency.
  - It has an Aggregate Root that controls access to the objects within it.
- 4. Domain Service
  - A Domain Service contains business logic that does not naturally belong to a particular entity or value object.
- 5. Factory
  - A Factory is responsible for creating complex domain objects or aggregates.
- 6. Repository
  - A Repository provides a mechanism for storing and retrieving domain objects.

# Transforming DDD into Microservices

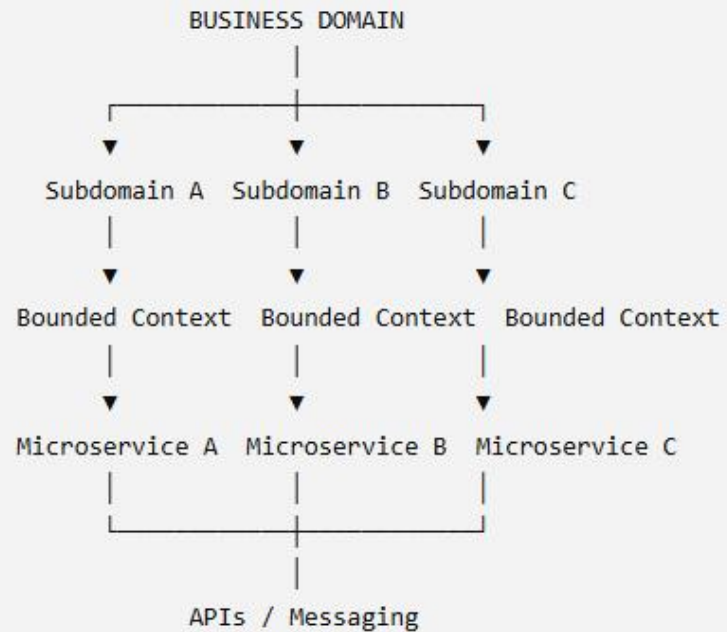
- Step 1: Define Bounded Contexts
  - Identify the major bounded contexts in the application.
  - Business Domain
  - ↓
  - Bounded Contexts
- Step 2: Analyze Each Context
- Identify:
  - Entities
  - Aggregates
  - Domain Services
- within each bounded context.

# Transforming DDD into Microservices

- Step 3: Define Services
  - Create services around the appropriate domain boundaries.
    - Bounded Context
      - ↓
      - Related Functions
        - ↓
        - Microservice
- Step 4: Establish Loose Coupling
  - Services communicate with each other while keeping their internal implementation independent.
- Step 5: Independent Services
  - Each service becomes a relatively small application focused on a specific business capability.

# DDD TO MSA DIAGRAM

This is a good diagram for examination:



**Remember:**

**Domain → Subdomain → Bounded Context → Microservice**

This is the easiest chain to remember for the examination.

# Real World Example

- Consider an online shopping application.

- Domain

- E-Commerce

- Subdomains

- Product Catalog
  - Inventory
  - Order Management
  - Delivery

## Bounded Contexts

Product Context

Inventory Context

Order Context

Delivery Context

## Microservices

Product Service  
Inventory Service  
Order Service  
Delivery Service

# Pros of DDD

- 1. Clear Service Boundaries
- 2. Business Alignment
- 3. Reduced Coupling
- 4. Better Communication
- 5. Better Maintainability
- 6. Stable Architecture

# Cons

- It requires strong understanding of the business.
- Identifying correct subdomains can be difficult.
- Defining bounded contexts requires careful analysis.
- Requires continuous collaboration between developers and domain experts.
- It can be complex for small/simple applications.

# Developing MSA Based Application

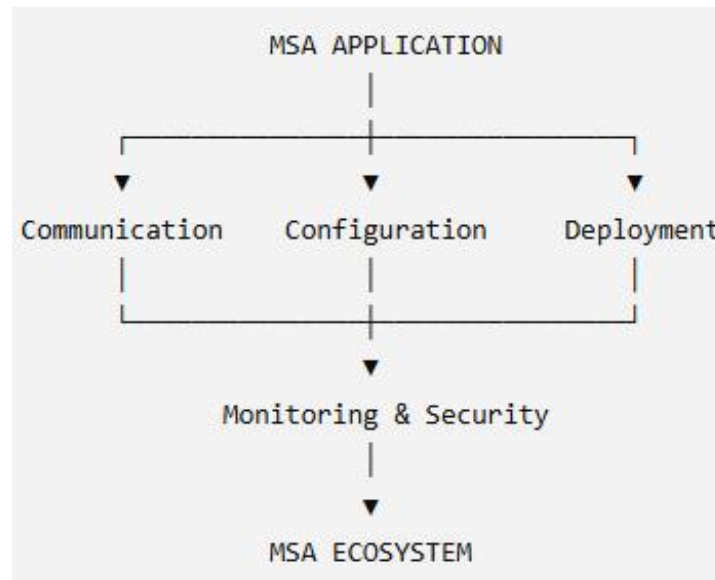
Understanding of Theory + Practical Aspects

## 1.3.1 Ecology for MSA

- Ecology for Microservices Architecture (MSA) refers to the supporting ecosystem of tools, technologies, infrastructure, and practices required to develop, run, communicate, monitor, and manage microservices.
- A microservice application is not just a collection of services. It requires an ecosystem around those services for communication, configuration, deployment, monitoring, security, and fault handling.

# Need for MSA Ecology

- In a monolithic application, many functions are managed within one application. In MSA, there may be many independent services running on different instances.
- Therefore, additional supporting components are required.



# Major Components of MSA Ecology

- 1. Microservices
  - The core components of the architecture.
  - Each service performs a specific business responsibility and can be independently developed and deployed.
- 2. API / Communication
  - Microservices need mechanisms to communicate with each other.
  - Common approaches include:
    - REST APIs
    - HTTP
    - Messaging
  - Your MSA material identifies REST APIs as well-defined interfaces for microservices.

- 3. Configuration Management
- Microservices require configuration such as:
  - Database details
  - Service URLs
  - Environment settings
  - Security settings
- 4. Service Discovery
- In a distributed environment, service instances may change their locations dynamically.
- A service registry can maintain the locations of available service instances so that clients can discover them. The reference book describes this as a key requirement in modern containerized/virtualized microservice environments.
- 5. Deployment Infrastructure
- Microservices need infrastructure for independent deployment and execution.
- Examples include:
  - Containers
  - Application servers
  - Cloud infrastructure
  - Microservers

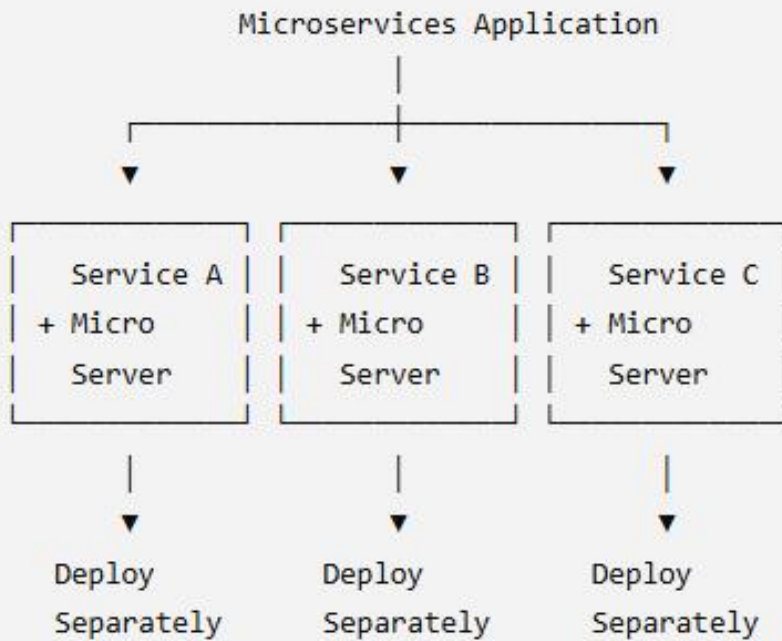
- 6. Monitoring and Observability
  - Because requests can pass through multiple services, the system needs monitoring mechanisms.
  - The MSA presentation includes an observability stack involving tools such as Prometheus, Jaeger, and Grafana.
- 7. Security
  - Authentication, authorization, and secure communication are required when services communicate with each other.
- 8. Fault Handling
  - Microservices should be resilient to failures. Mechanisms such as Circuit Breakers and Bulkheads can help prevent failures from spreading across services.

## 1.3.2 Micro Servers

- A Micro Server is a lightweight server/runtime environment designed to run a microservice with minimal overhead. It supports the principle that each microservice should be small, self-contained, and independently deployable.
- Key Characteristics
  - Lightweight – Requires fewer system resources.
  - Fast Startup – Starts quickly because it contains only the required functionality.
  - Self-Contained – A service can carry the runtime components it needs.
  - Independent Deployment – Each microservice can be deployed separately.
  - Easy to Scale – Multiple instances of a service can be created when required.
  - Technology Flexibility – Different services can use suitable technologies/runtime environments.

# Diagram

## Micro Server in MSA



The idea is that instead of deploying the complete application on one large application server, **each microservice can run in its own lightweight runtime environment.**

# Why Micro Servers are Used?

- Microservices require frequent and independent deployment. A heavyweight application server can introduce unnecessary overhead. A lightweight micro-server/runtime provides only what the service requires, helping achieve:
  - Faster startup
  - Lower resource consumption
  - Easier deployment
  - Better scalability
  - Greater service independence

## Micro Server vs Traditional Application Server

### Micro Server

Lightweight

Focused on a service

Fast startup

Suitable for independent microservices

Smaller resource footprint

Supports independent deployment

### Traditional Application Server

Generally heavyweight

Can host a large application

Relatively larger startup overhead

Commonly suited to larger applications

Higher resource requirements

Often associated with application-level deployment

## 1.3.3 REST API for MSA

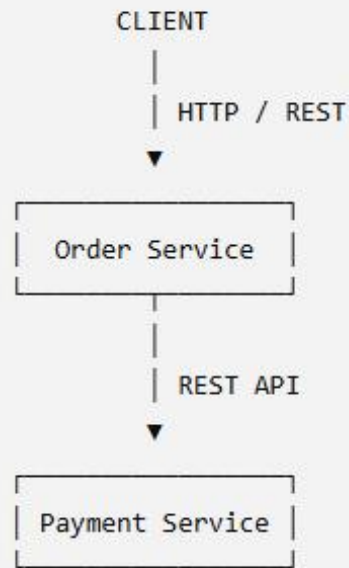
- REST API is one of the most important communication mechanisms used in Microservices Architecture. Your MSA PPT specifically states that microservices communicate through REST API calls and that each microservice has a well-defined REST API interface.
- What is REST?
- REST (Representational State Transfer) is an architectural style for designing web-based services using standard HTTP operations.
- A REST API allows a client to access and manipulate resources provided by a service.

Simple Example

Client --> HTTP Request --> Order Service --> HTTP Response --> Client

# REST API in Microservices

In MSA, each microservice exposes its functionality through a well-defined API.



For example:

- **Customer Service** → Customer REST API
- **Order Service** → Order REST API
- **Payment Service** → Payment REST API
- **Inventory Service** → Inventory REST API

# Rivision

## 3. HTTP Methods Used by REST API

REST commonly uses HTTP methods to perform operations on resources.

| HTTP Method | Purpose             | Example      |
|-------------|---------------------|--------------|
| GET         | Retrieve data       | Get customer |
| POST        | Create new data     | Create order |
| PUT         | Update/replace data | Update order |
| DELETE      | Delete data         | Delete order |

### Example

For an **Order Service**:

```
GET    /orders/101    → Get Order 101
POST   /orders       → Create an Order
PUT    /orders/101   → Update Order 101
DELETE /orders/101   → Delete Order 101
```



## 4. REST Resource

A **resource** is an object or business entity that can be accessed through a REST API.

Examples:

```
/customers  
/orders  
/products  
/payments  
/inventory
```



Each resource is identified using a **URI (Uniform Resource Identifier)**.

Example:

```
/orders/101
```



Here:

- `/orders` → Resource
- `101` → Specific order

## 5. Request and Response

REST communication generally follows:



### Example Response

`</>` JSON

```
{
  "orderId": 101,
  "customer": "Vraj",
  "status": "Confirmed"
}
```

JSON is commonly used because it is lightweight and easy for different applications to process.

## 6. REST API and Loose Coupling

REST APIs help maintain **loose coupling** between microservices.

The client only knows:

API  
↓  
Request  
↓  
Response



It does **not** need to know:

Internal Classes  
Internal Database  
Internal Business Logic  
Internal Implementation



Therefore:

Change Internal Implementation  
↓  
API unchanged  
↓  
Client unaffected

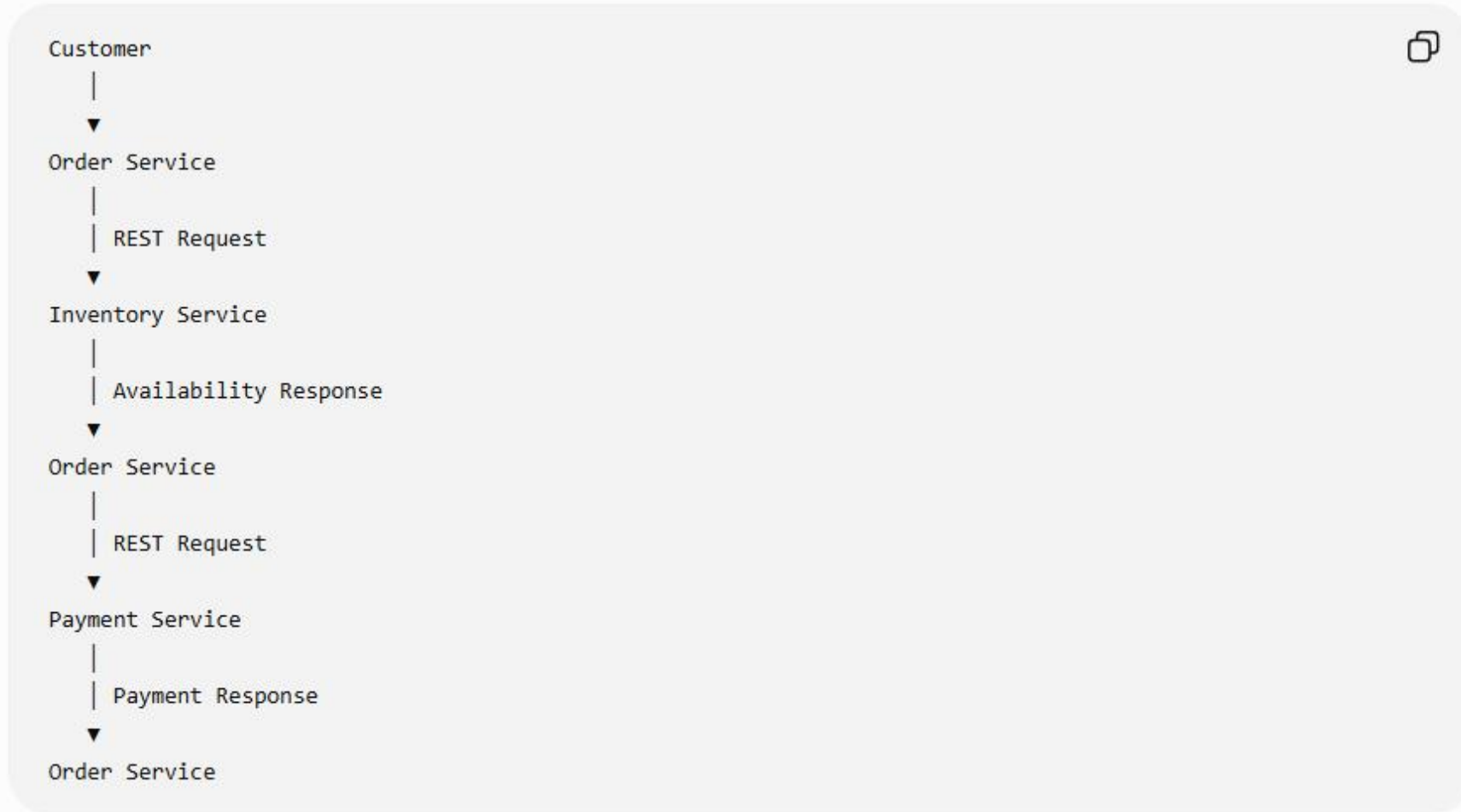


This supports one of the major goals of MSA: **independent development and deployment**.

The reference book emphasizes that a service's API should encapsulate its implementation so that the implementation can change without affecting clients.

## 7. REST Communication Between Microservices

Consider an online shopping process:



Thus, REST allows different microservices to communicate through standardized HTTP requests and responses.

Your PPT gives a cab application example where the monolith is decomposed into independent services running in containers and the services communicate through **REST API calls**.

# PROS

## 1. Loose Coupling

Services communicate through APIs instead of directly accessing each other's internal implementation.

## 2. Platform Independent

REST/HTTP communication can be used by applications written in different programming languages.

## 3. Simple

REST uses standard HTTP methods and is relatively easy to understand and implement.

## 4. Independent Development

Different teams can develop services independently as long as the API contract is maintained.

## 5. Independent Deployment

A service can be updated and deployed without rebuilding the entire application.

## 6. Client Independence

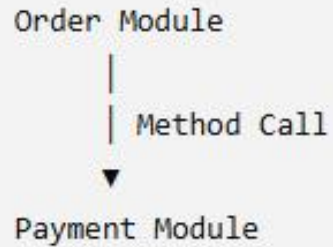
Different clients such as web applications, mobile applications, and other services can consume the REST API.

# LIMITATIONS

- Network latency
- Network failures
- Timeout handling
- More communication overhead
- API versioning requirements
- Distributed debugging
- Security between services

## 10. REST API vs Traditional Method Calls

### Monolithic Application



Communication happens inside the same application/process.

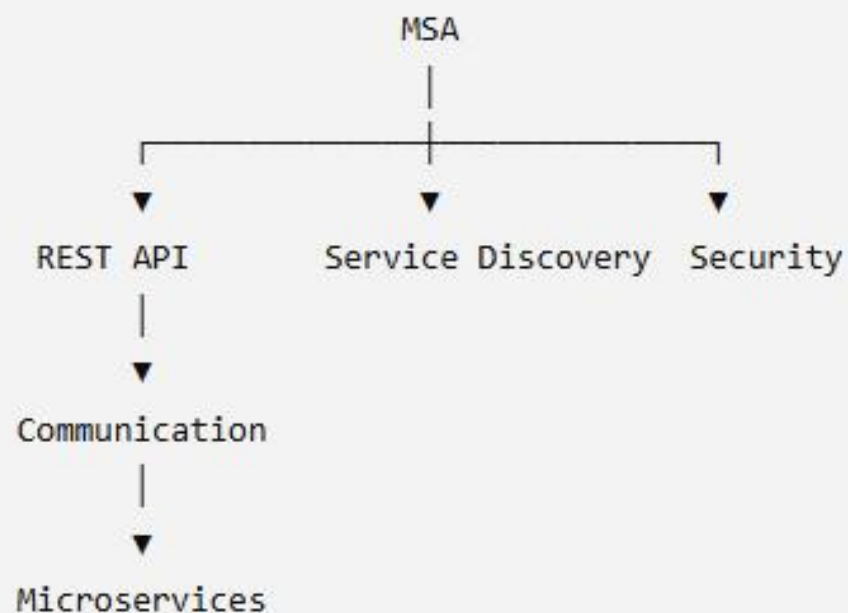
### Microservices



The services are separate applications and communicate over the network.

## 11. REST API in the MSA Ecosystem

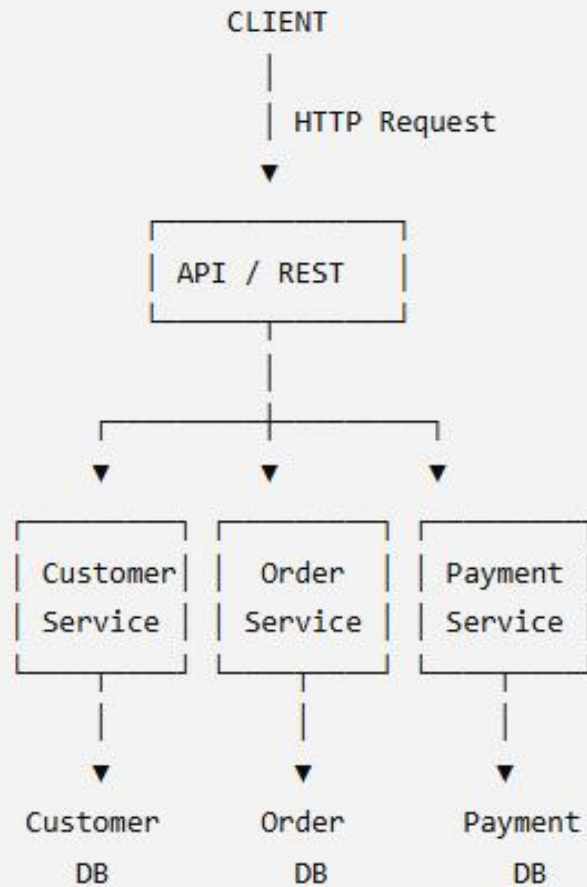
REST is only **one part** of the complete MSA ecosystem.



Other MSA concerns such as **API Gateway**, **service discovery**, **security**, **monitoring**, and **deployment** are covered separately in your syllabus/reference material.

## ★ 12. Important Exam Diagram

Draw this for a question such as “Explain REST API in Microservices Architecture.”

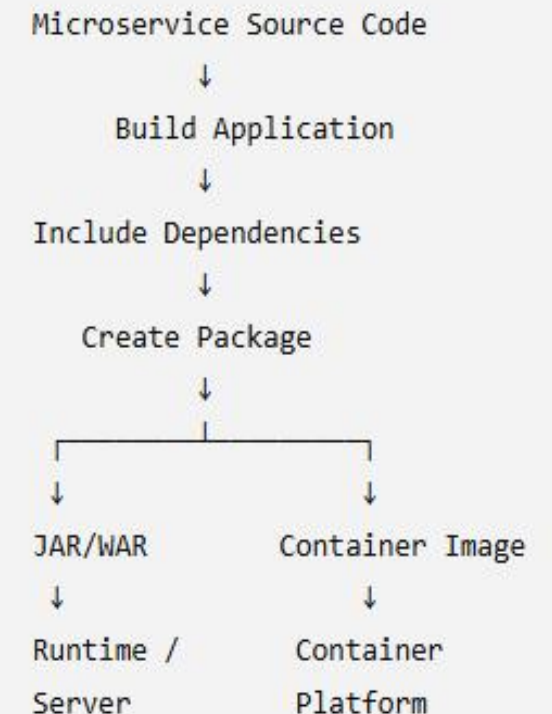


Services communicate using REST/HTTP

# 1.3.4 Packaging Microservices Applications

- Packaging is the process of preparing a microservice along with the required application files and dependencies so that it can be deployed and executed independently.
- In Microservices Architecture, each service should be packaged separately so that it can be developed, deployed, updated, and scaled independently.
- Common Packaging Approaches
  - JAR Packaging – A Java microservice and its required components can be packaged into a JAR file and executed with a suitable Java runtime.
  - WAR Packaging – A microservice can be packaged as a WAR file when it is intended to run on an application server.
  - Container Packaging – A microservice and its runtime dependencies can be packaged into a container image, allowing it to run consistently across different environments.

## Packaging Process

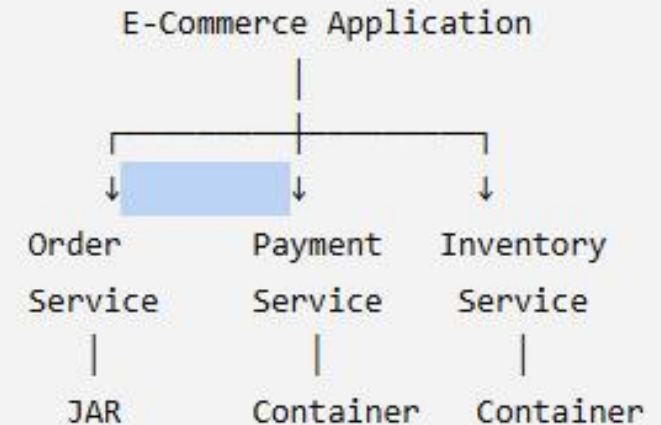


# Why Packaging is Important?

- Independent Deployment: Each microservice can be deployed separately.
- Portability: The packaged service can be moved between environments.
- Consistency: Required dependencies and runtime configuration can be managed with the application.
- Easy Scaling: Multiple instances of the same packaged service can be deployed.
- Simplified Management: Each service has its own deployable unit.

## Example

Consider an e-commerce application:



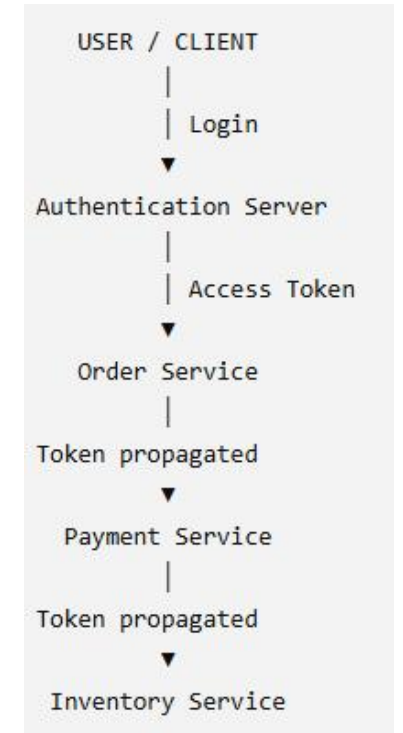
## 1.3.5 Security Propagation across Micro Services.

- Security Propagation means carrying the security information of a user/request from one microservice to another when multiple services participate in processing the same request.
- In a monolithic application, security can often be handled centrally. In MSA, services are independent, so the security context must be appropriately propagated across service boundaries. Your PPT lists “Central Security Control and Propagation” among the traditional enterprise-application mechanisms and identifies JWT Authentication as a MicroProfile capability.

# Why Security Propagation is Required?

- Consider an application where a user requests an order:
- User
- ↓
- Order Service
- ↓
- Payment Service
- ↓
- Inventory Service

Basic Flow -->



- The Payment and Inventory services should know that the request is coming from an authenticated and authorized user.
- Therefore, security information needs to propagate along with the request.

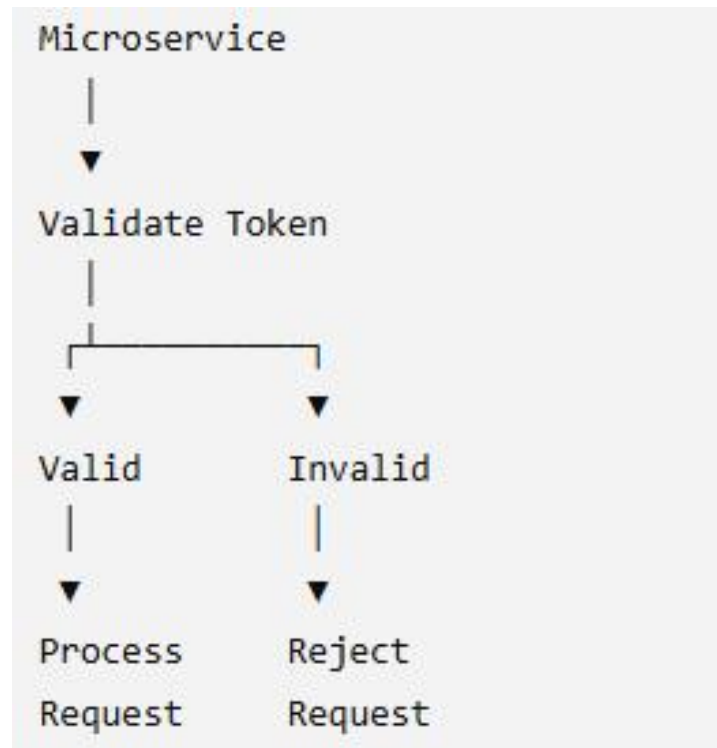
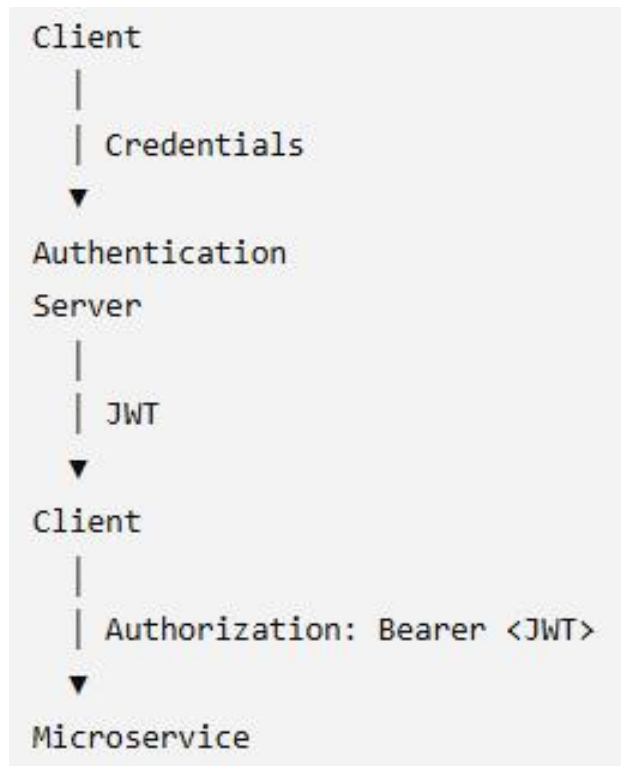
# Authentication and Authorization

- Authentication
  - Authentication verifies who the user is.
- Authorization
  - Authorization determines what the authenticated user is allowed to do.
- Token-Based Security



# JWT Authentication

- JWT (JSON Web Token) is a token format commonly used for transmitting authentication and authorization information.



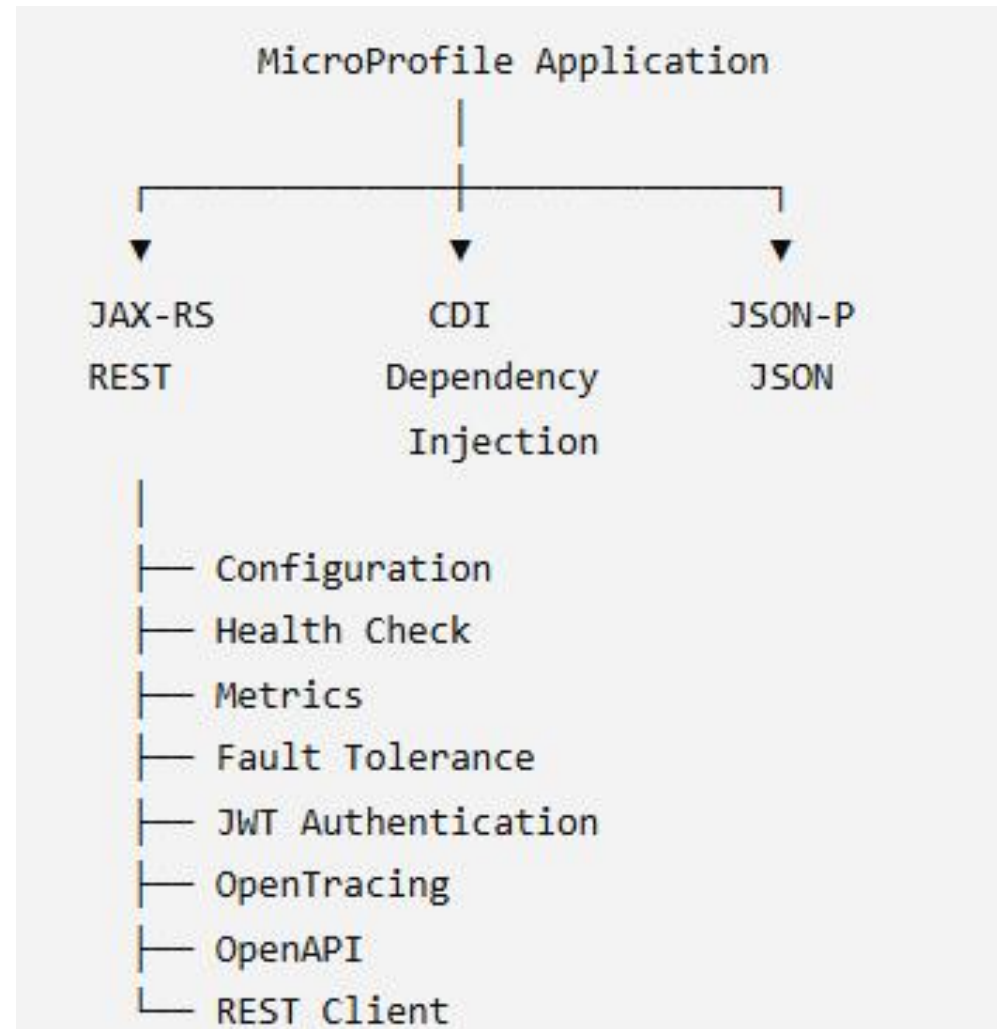
# Propagation Across Multiple Services

- Step 1 – User Authentication
  - The user authenticates and receives a token.
- Step 2 – Request to Order Service
  - The client sends the token with the request.
- Step 3 – Order Service Validation
  - The Order Service validates the token and processes the request.
- Step 4 – Call to Payment Service
  - When Order Service calls Payment Service, the required security information can be propagated.
- Step 5 – Downstream Validation
  - Payment Service validates the security information before processing the request.

# 1.3.6 Developing MicroProfile-based Application for MSA

- MicroProfile is a Java-based platform designed to provide the required programming models and APIs for developing cloud-native and microservices-based applications.
- MicroProfile extends commonly used Java enterprise technologies with features specifically useful for microservices.
- Major MicroProfile Features
- According to your PPT:
  - JAX-RS – REST
  - CDI
  - JSON-P
  - Common Annotations
  - Config
  - Metrics
  - Health Check
  - Fault Tolerance
  - JWT Authentication
  - OpenTracing
  - OpenAPI
  - REST Client

# Basic Architecture



# Important Features

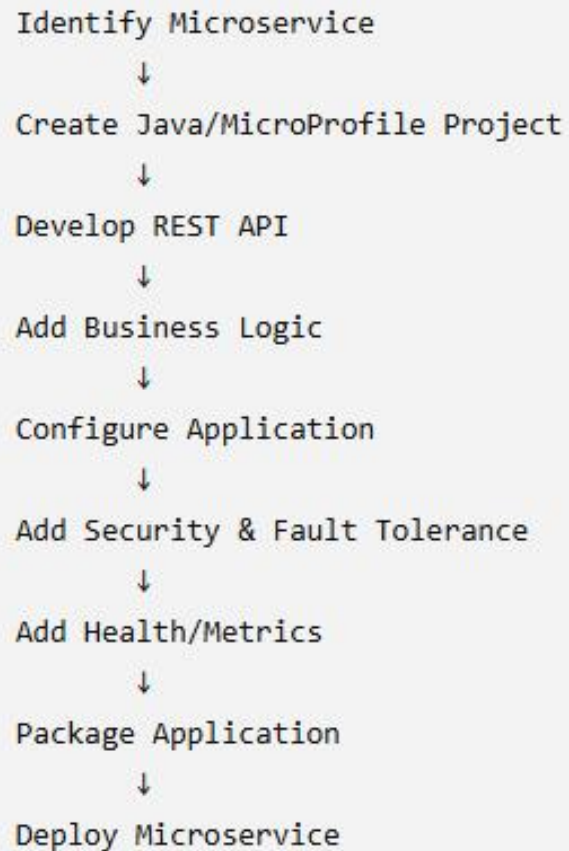
- 1. JAX-RS – REST
  - Used to develop RESTful web services through which microservices communicate.
- 2. CDI
  - Contexts and Dependency Injection (CDI) helps manage dependencies between application components.
- 3. JSON-P
  - Provides APIs for processing JSON data, which is commonly exchanged between microservices.
- 4. MicroProfile Config
  - Allows application configuration to be managed separately from application code. This is useful when the same service is deployed in different environments.
- 5. Metrics
  - Provides application metrics that help in monitoring and analyzing microservices.
- 6. Health Check
  - Allows the health/status of a deployed microservice to be checked.
  - Your PPT demonstrates these through MicroProfile Fault Tolerance annotations such as `@Retry`, `@Fallback`, `@Timeout`, `@CircuitBreaker`, and `@Bulkhead`

# Important Features

- 8. JWT Authentication
  - JWT (JSON Web Token) Authentication provides a mechanism for securing microservices using tokens. It is specifically listed among the MicroProfile features in your PPT.
- 9. OpenTracing
  - Used for distributed tracing, helping track a request as it moves through multiple microservices.
- 10. OpenAPI
  - Used to describe and document REST APIs.
- 11. REST Client
  - Provides a standardized way for one microservice to invoke another REST-based service

# 3. Developing a MicroProfile Application

A simplified development process is:

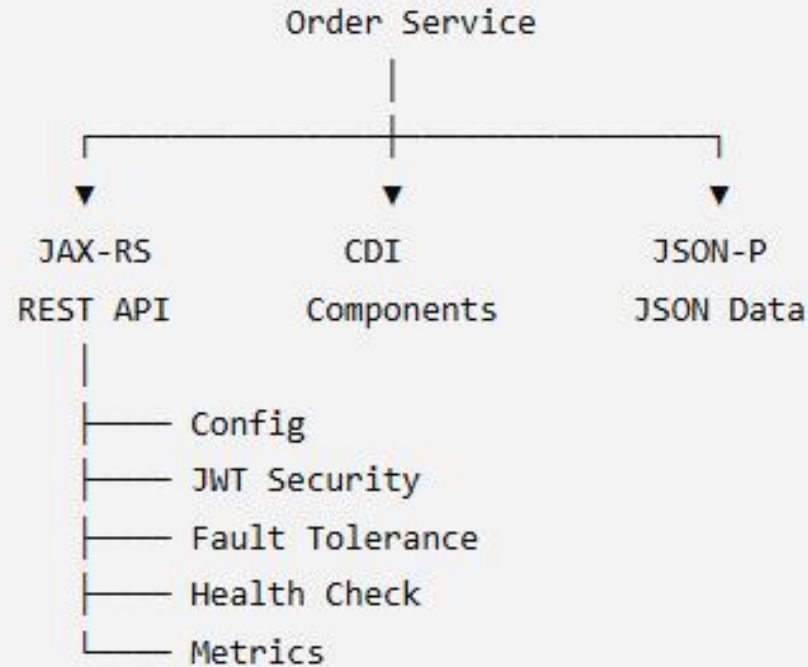


# Why MicroProfile for MSA?

- MicroProfile is designed to address important requirements of microservices applications.
- Continuous delivery and deployment
- Improved maintainability
- Better testability
- Independent deployment
- Improved fault isolation
- Faster application startup
- Easier understanding for developers

## 5. Example

Consider an **Order Service** developed using MicroProfile.

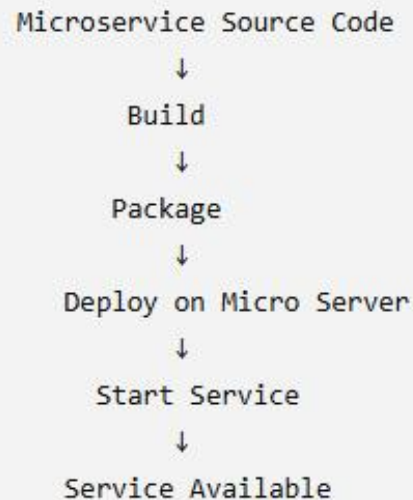


The service can expose REST APIs, use external configuration, secure requests with JWT, handle failures, and expose health/monitoring information.

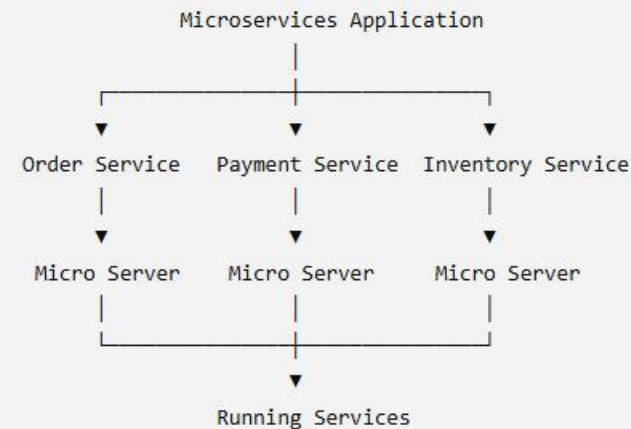
## 1.3.7 Deploying Applications on Micro Servers

- Deployment is the process of making a microservice available to run in its target environment. In MSA, each microservice is packaged and deployed independently on a suitable lightweight runtime or micro server.

### Basic Deployment Process



### Deployment Architecture



# STEPS TO DEPLOY

- 1. Develop the Microservice
  - Create the service and implement its required business functionality and APIs.
- 2. Build the Application
  - Compile the source code and prepare the application for execution.
- 3. Package the Service
  - Create a deployable package such as a JAR, WAR, or container image, depending on the deployment environment.
- 4. Deploy on Micro Server
  - Place the packaged microservice on the required micro server/runtime.
- 5. Configure the Service
  - Provide required configuration such as:
    - Port
    - Database connection
    - Service endpoints
    - Environment-specific settings
- 6. Start and Monitor
  - Start the service and monitor its health, performance, and availability.

# Thank You

Vraj Suratwala

Department of ICT, VNSGU